

# طراحی الگوریتم‌ها – تقسیم و حل

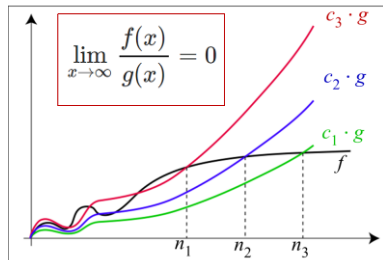
Introduction to Algorithm

مهدی جوانمردی

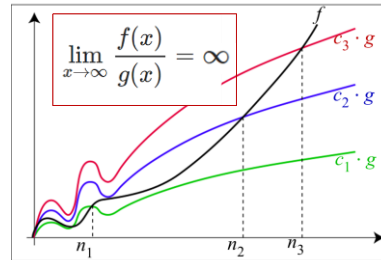
# مرور مباحث قبل

## نمادهای $O$ و $\omega$ کوچک

The  $o$ -Notation



The  $\omega$ -Notation



الگوریتم A بهتر است الگوریتم B است ←  $T_A(n) = o(T_B(n))$

## خواص توابع و نمادهای استاندارد

تکرار تابع

Functional Iteration

$$f^{(i)}(n) = \begin{cases} n & \text{if } i = 0, \\ f(f^{(i-1)}(n)) & \text{if } i > 0. \end{cases}$$

لگاریتم استار

Iterated logarithmic function

$$\lg^* n = \min \{i \geq 0 : \lg^{(i)} n \leq 1\}$$

تراگذری

Transitivity

بازتابی

Reflexivity

یکنوایی تابع

Monotonicity

تقارنی

Symmetry

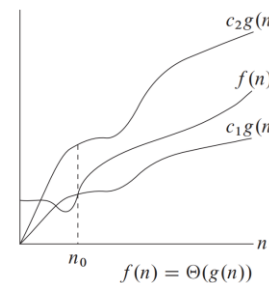
ضد تقارنی

Transpose Symmetry

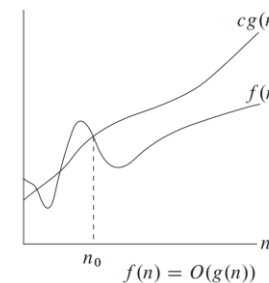
## نمادهای تقریبی رشد توابع

| Name        | Notation             | $f(n) = O(g(n)) \approx a \leq b,$      |
|-------------|----------------------|-----------------------------------------|
| Big O       | $\mathcal{O}$ or $O$ | $f(n) = \Omega(g(n)) \approx a \geq b,$ |
| Big Omega   | $\Omega$             | $f(n) = \Theta(g(n)) \approx a = b,$    |
| Big Theta   | $\Theta$             | $f(n) = o(g(n)) \approx a < b,$         |
| Small O     | $o$                  | $f(n) = \omega(g(n)) \approx a > b.$    |
| Small Omega | $\omega$             |                                         |

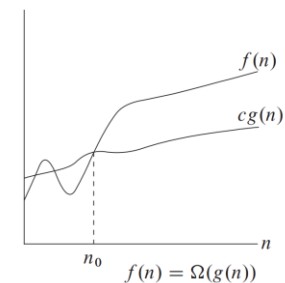
## رشد توابع و حد بینهایت



$$\lim_{x \rightarrow \infty} \frac{f(x)}{g(x)} = c$$



$$\lim_{x \rightarrow \infty} \frac{f(x)}{g(x)} = c \text{ or } 0$$



$$\lim_{x \rightarrow \infty} \frac{f(x)}{g(x)} = \infty \text{ or } c$$

# فصل چهارم: تقسیم و حل | Divide and Conquer

- مسئله زیرآرایه بیشینه یا Maximum Subarray

- انواع روش‌های حل رابطه بازگشت

- جایگذاری و استقرای ریاضی

- درخت بازگشت

- قضیه اصلی

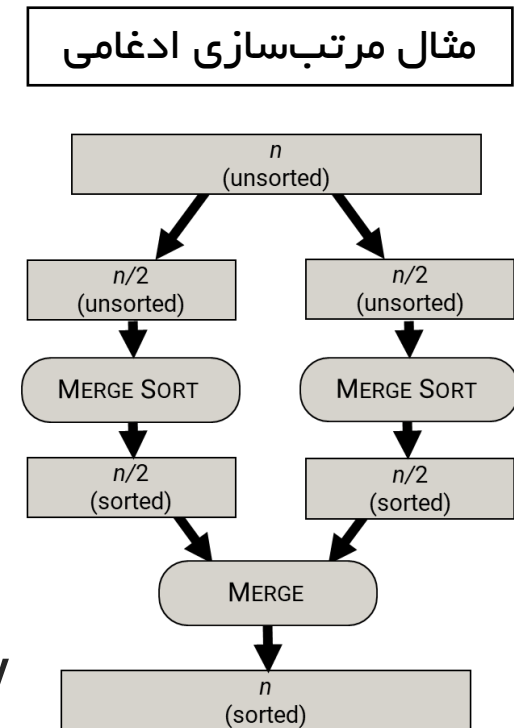
- الگوریتم استراسن برای ضرب ماتریسی

## 4 Divide-and-Conquer 65

- 4.1 The maximum-subarray problem 68
- 4.2 Strassen's algorithm for matrix multiplication 75
- 4.3 The substitution method for solving recurrences 83
- 4.4 The recursion-tree method for solving recurrences 88
- 4.5 The master method for solving recurrences 93
- ★ 4.6 Proof of the master theorem 97

# الگوریتم‌های تقسیم و حل

- **The divide-and-conquer approach:**
  - **Divide** the problem into a number of subproblems  
(that are smaller instances of the same problem)
  - **Conquer** the subproblems by solving them recursively
    - **Base case:** If the subproblem sizes are small enough, solve it in a straightforward manner
  - **Combine** the solutions to solve the original problem
- **Recursive structure**
  - To solve a given problem, they call themselves recursively one or more times to deal with related subproblems



نکته:

بعضی از الگوریتم‌های حل و تقسیم، در کنار حل زیرمسئله‌های مشابه کوچکتر، لازم است زیرمسئله‌هایی که خیلی تشابهی با مسئله اصلی ندارند را نیز حل کنیم!

بخشی از مرحله ترکیب

# مروری بر رابطه بازگشتی

Recurrences go hand in hand with the divide-and-conquer paradigm because they give us a natural way to characterize the running times of divide-and-conquer algorithms.

- A **recurrence** is a function defined in terms of
  - One or more base cases, and
  - Itself, with smaller arguments.

## Examples

$$T(n) = \begin{cases} 1 & \text{if } n = 1, \\ T(n-1) + 1 & \text{if } n > 1. \end{cases}$$

Solution:  $T(n) = n$ .

$$T(n) = \begin{cases} 0 & \text{if } n = 2, \\ T(\sqrt{n}) + 1 & \text{if } n > 2. \end{cases}$$

Solution:  $T(n) = \lg \lg n$ .

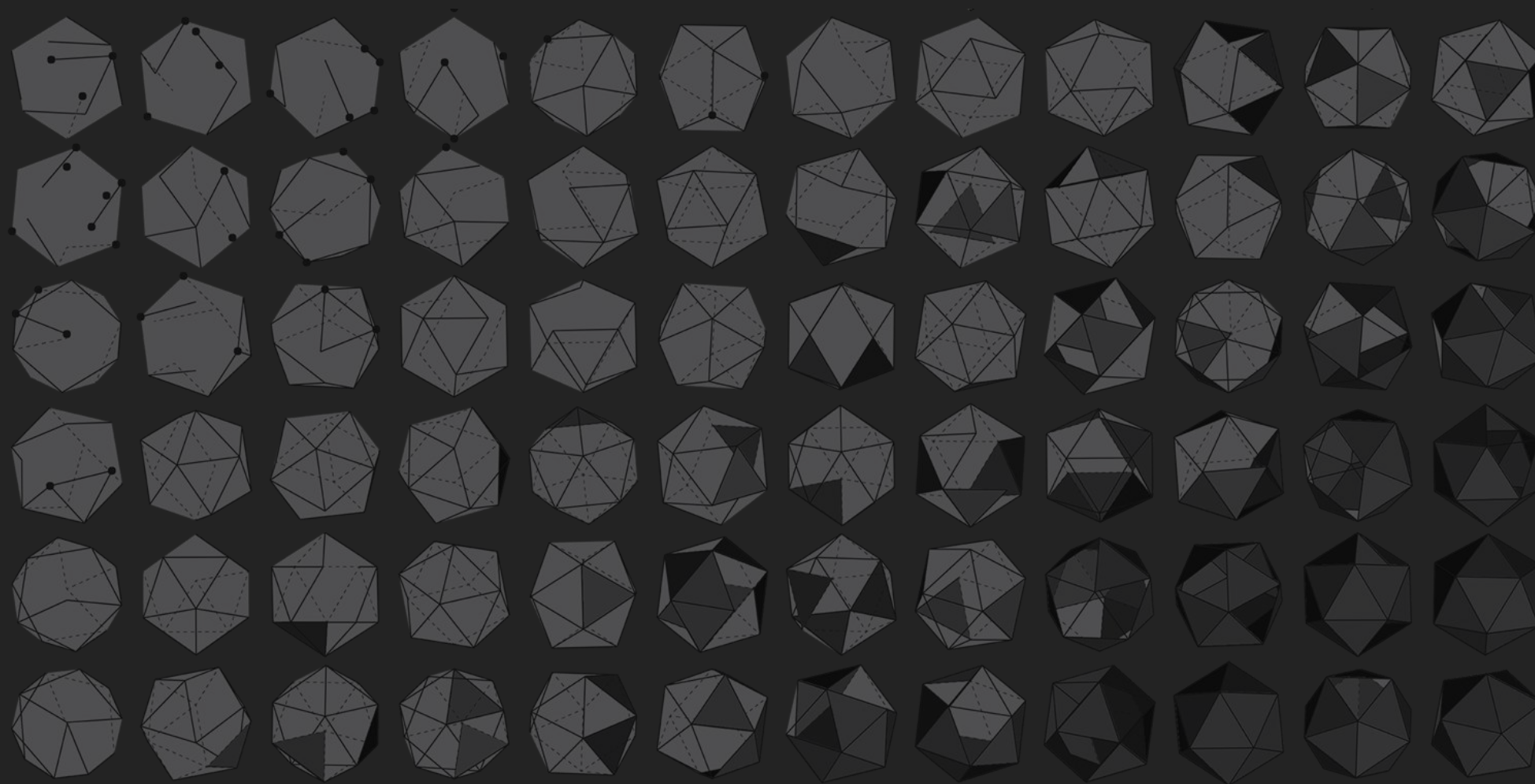
$$T(n) = \begin{cases} 1 & \text{if } n = 1, \\ 2T(n/2) + n & \text{if } n \geq 2. \end{cases}$$

Solution:  $T(n) = n \lg n + n$ .

$$T(n) = \begin{cases} 1 & \text{if } n = 1, \\ T(n/3) + T(2n/3) + n & \text{if } n > 1. \end{cases}$$

Solution:  $T(n) = \Theta(n \lg n)$ .





# مسئله زیر آرایه بیشینه یا Maximum Subarray

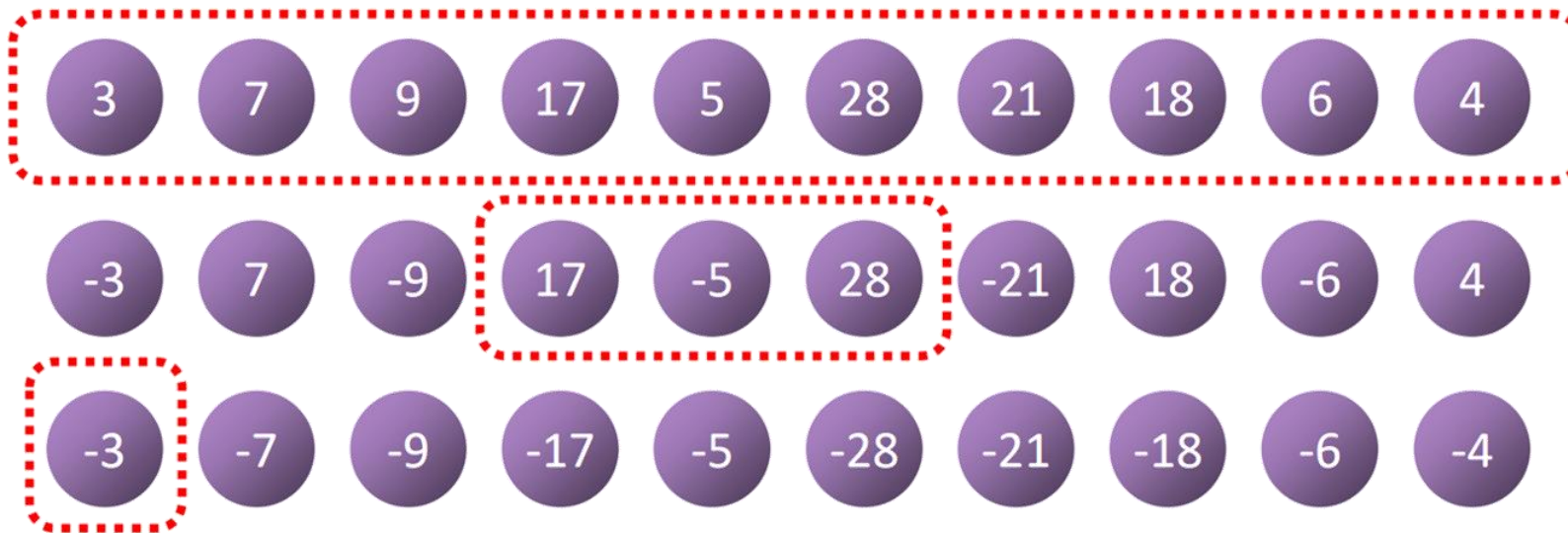
فصل ۴.۱ کتاب

# مسئله زیرآرایه بیشینه یا Maximum Subarray

**Input:** An array  $A[1..n]$  of numbers. *[Assume that some of the numbers are negative, because this problem is trivial when all numbers are nonnegative.]*

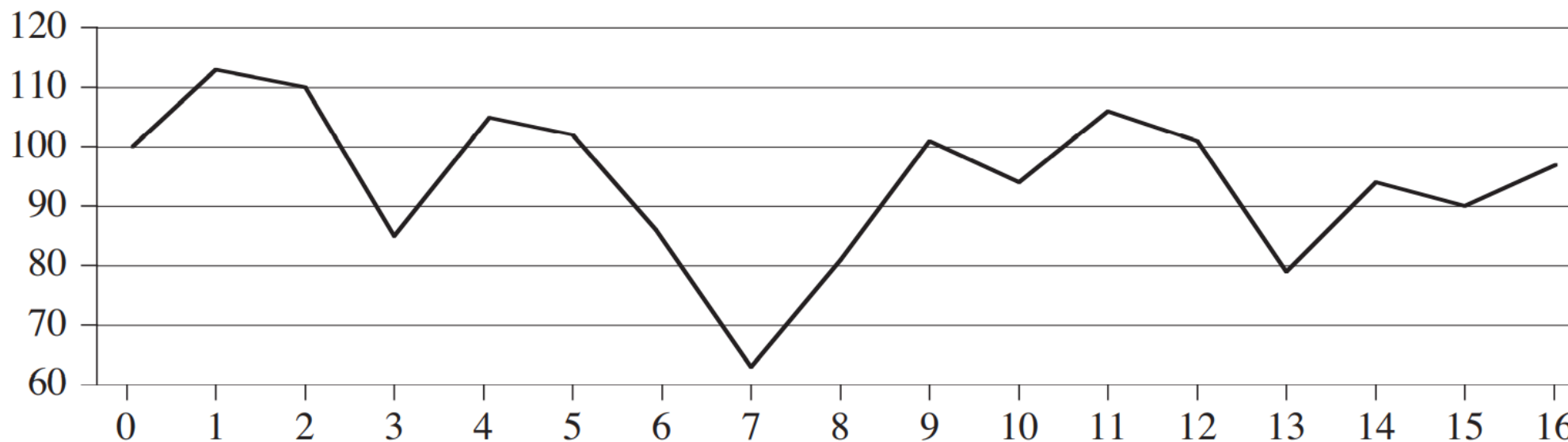
**Output:** Indices  $i$  and  $j$  such that  $A[i..j]$  has the greatest sum of any nonempty, contiguous subarray of  $A$ , along with the sum of the values in  $A[i..j]$ .

$$A[i] + A[i + 1] + \dots + A[j].$$



# کسب حداکثر سود در خرید و فروش سهام

تغییرات سهام شرکت الف در دوره زمانی ۱۷ روزه



| Day   | 0   | 1   | 2   | 3  | 4   | 5   | 6  | 7  | 8  | 9   | 10 | 11  | 12  | 13 | 14 | 15 | 16 |
|-------|-----|-----|-----|----|-----|-----|----|----|----|-----|----|-----|-----|----|----|----|----|
| Price | 100 | 113 | 110 | 85 | 105 | 102 | 86 | 63 | 81 | 101 | 94 | 106 | 101 | 79 | 94 | 90 | 97 |

- You have the prices that a stock traded at over a period of  $n$  consecutive days.
- When should you have bought the stock? When should you have sold the stock?

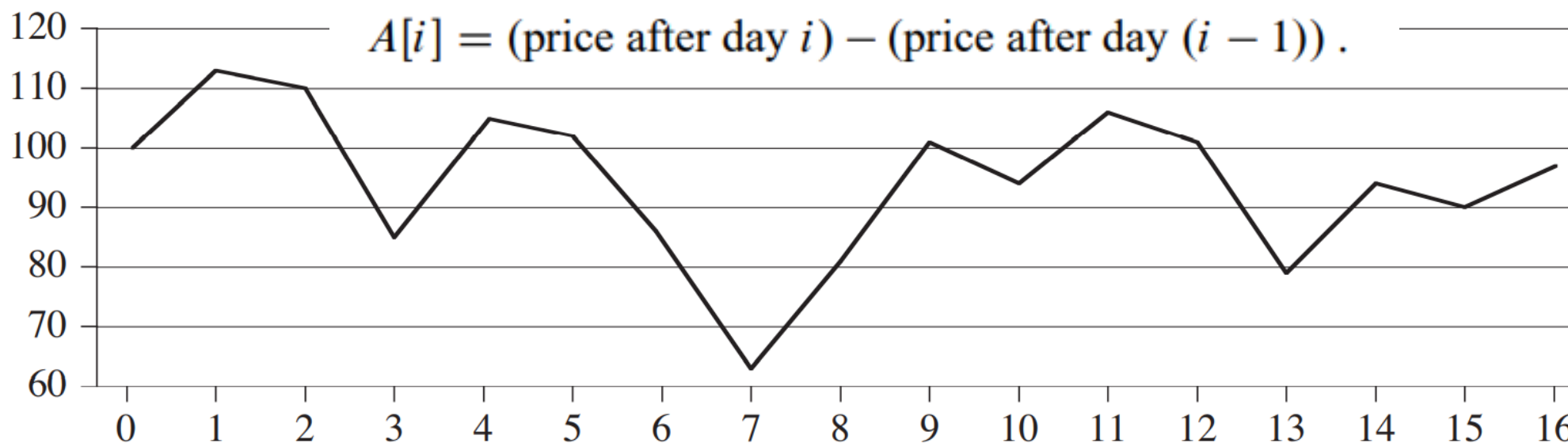
**Goal: Maximize your profit**



# کسب حداکثر سود در خرید و فروش سهام

To convert to a maximum-subarray problem, let

$$A[i] = (\text{price after day } i) - (\text{price after day } (i - 1)) .$$



|        |     |     |     |     |     |     |     |     |    |     |    |     |     |     |    |    |    |
|--------|-----|-----|-----|-----|-----|-----|-----|-----|----|-----|----|-----|-----|-----|----|----|----|
| Day    | 0   | 1   | 2   | 3   | 4   | 5   | 6   | 7   | 8  | 9   | 10 | 11  | 12  | 13  | 14 | 15 | 16 |
| Price  | 100 | 113 | 110 | 85  | 105 | 102 | 86  | 63  | 81 | 101 | 94 | 106 | 101 | 79  | 94 | 90 | 97 |
| Change |     | 13  | -3  | -25 | 20  | -3  | -16 | -23 | 18 | 20  | -7 | 12  | -5  | -22 | 15 | -4 | 7  |

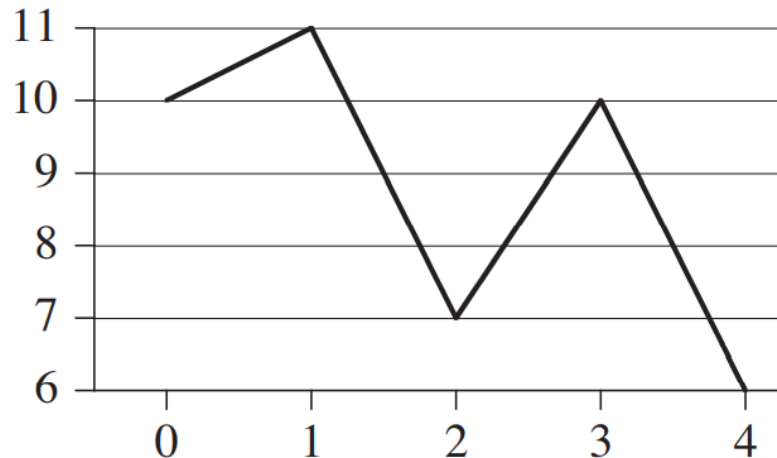
Goal: Maximize your profit



“buy low, sell high” ??

# کسب حداکثر سود در خرید و فروش سهام

- Lowest price might occur *after* the highest price.
- But wouldn't the optimal strategy involve buying at the lowest price *or* selling at the highest price?
- Not necessarily:



| Day    | 0  | 1  | 2  | 3  | 4  |
|--------|----|----|----|----|----|
| Price  | 10 | 11 | 7  | 10 | 6  |
| Change |    | 1  | -4 | 3  | -4 |

# حل زیرآرایه بیشینه با روش تهاجمی

## Brute Force Algorithm

تاریخ خرید سهام

|             | Sea | LA | Den | Dal | StP | Atl | Cin | NY | NO | Mia |
|-------------|-----|----|-----|-----|-----|-----|-----|----|----|-----|
| Seattle     | 0   | 5  | 5   | 8   | 7   | 11  | 10  | 12 | 10 | 13  |
| Los Angeles | 5   | 0  | 4   | 6   | 7   | 9   | 9   | 12 | 8  | 11  |
| Denver      | 5   | 4  | 0   | 3   | 3   | 6   | 5   | 8  | 5  | 8   |
| Dallas      | 8   | 6  | 3   | 0   | 4   | 3   | 4   | 7  | 2  | 5   |
| St. Paul    | 7   | 7  | 2   | 4   | 0   | 4   | 2   | 6  | 5  | 7   |
| Atlanta     |     |    |     |     |     |     |     |    |    |     |
| Cincinnati  | 10  | 9  | 5   | 4   | 3   | 2   | 0   | 3  | 4  | 3   |
| New York    | 12  | 12 | 8   | 7   | 5   | 4   | 3   | 0  | 6  | 5   |
| New Orleans | 10  | 8  | 5   | 2   | 5   | 2   | 4   | 6  | 0  | 3   |
| Miami       | 13  | 11 | 8   | 5   | 7   | 3   | 3   | 5  | 3  | 0   |

Can solve by brute force: check all  $\binom{n}{2} = \Theta(n^2)$  subarrays.

تاریخ فروش سهام

بیشینه سود حاصله

11

# حل زیرآرایه بیشینه با روش تهاجمی $O(n^3)$

## $O(n^3)$ Brute Force Algorithm

```

MaxSubarray-1(i, j)
  for i = 1, ..., n
    for j = 1, ..., n
      S[i][j] = -∞

  for i = 1, ..., n
    for j = i, i+1, ..., n
      S[i][j] = A[i] + A[i+1] + ... + A[j]

  return Champion(S)

```

$O(n^2)$

$\left. \begin{array}{l} \text{for } i = 1, \dots, n \\ \text{for } j = i, i+1, \dots, n \end{array} \right\} O(n^3)$

$O(n^2)$

|   |    |    |     |    |    |     |     |    |    |    |    |    |     |    |    |    |
|---|----|----|-----|----|----|-----|-----|----|----|----|----|----|-----|----|----|----|
|   | 1  | 2  | 3   | 4  | 5  | 6   | 7   | 8  | 9  | 10 | 11 | 12 | 13  | 14 | 15 | 16 |
| A | 13 | -3 | -25 | 20 | -3 | -16 | -23 | 18 | 20 | -7 | 12 | -5 | -22 | 15 | -4 | 7  |

# حل زیرآرایه بیشینه با روش تهاجمی $O(n^2)$

## $O(n^2)$ Brute Force Algorithm

```

MaxSubarray-2(i, j)
  for i = 1, ..., n
    for j = 1, ..., n
      S[i][j] = -∞

  R[0] = 0
  for i = 1, ..., n
    R[i] = R[i-1] + A[i]

  for i = 1, ..., n
    for j = i+1, i+2, ..., n
      S[i][j] = R[j] - R[i-1]

  return Champion(S)
  
```

$O(n^2)$

$R[n]$  is the sum over  $A[1..n]$  }  $O(n)$

}  $O(n^2)$

$O(n^2)$

|   |    |    |     |    |    |     |     |    |    |    |    |    |     |    |    |    |
|---|----|----|-----|----|----|-----|-----|----|----|----|----|----|-----|----|----|----|
|   | 1  | 2  | 3   | 4  | 5  | 6   | 7   | 8  | 9  | 10 | 11 | 12 | 13  | 14 | 15 | 16 |
| A | 13 | -3 | -25 | 20 | -3 | -16 | -23 | 18 | 20 | -7 | 12 | -5 | -22 | 15 | -4 | 7  |



# حل زیرآرایه بیشینه با روش تقسیم و حل

آیا روشی با مرتبه زمانی بهتر برای این مسئله می‌توان پیدا کرد؟

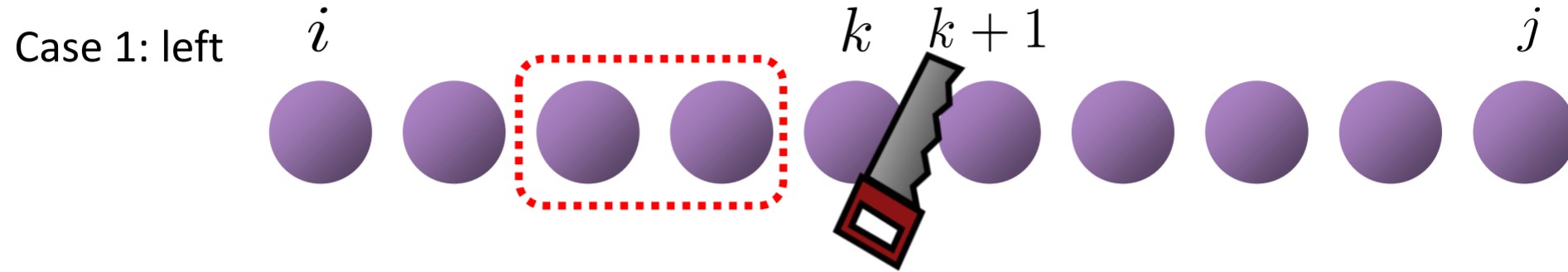
**نکته:** روش تهاجمی قادر به کشف همه MSA بود درحالیکه ما معمولاً نیاز به صرفا یکی از MSAها داریم و نه همه آنها

## روش تقسیم حل برای MSA

- Base case ( $n = 1$ )
  - Return itself (maximum subarray)
- Recursive case ( $n > 1$ )
  - Divide the array into two sub-arrays
  - Find the maximum sub-array recursively
  - Merge the results **How?**

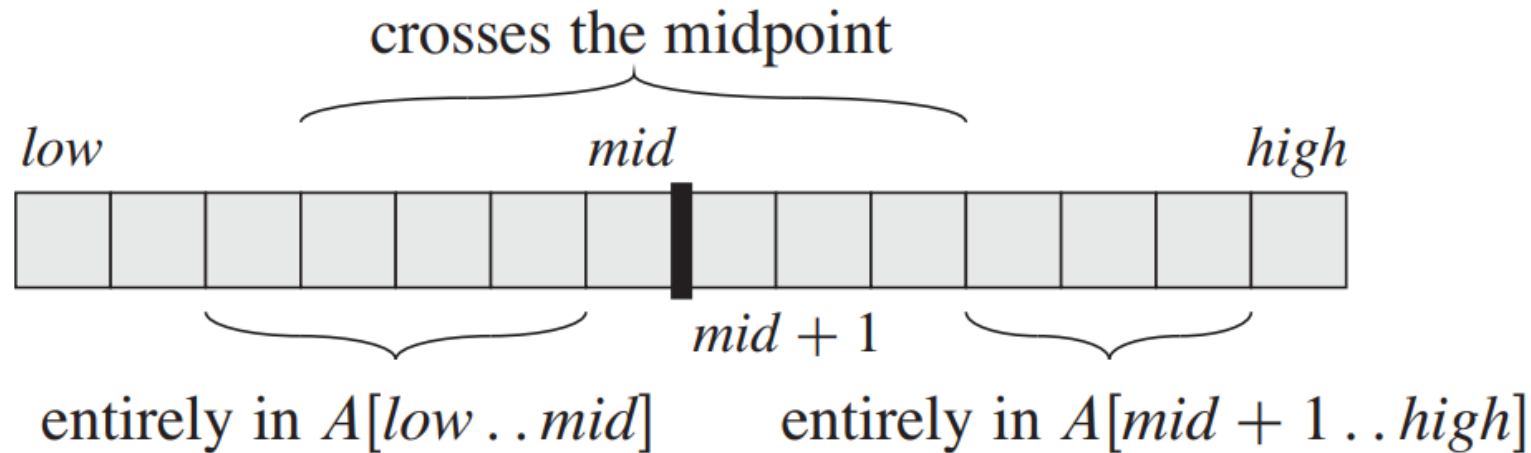
# جواب مسئلہ کجاست؟

- The maximum subarray for any input must be in one of following cases:



# جواب مسئله کجاست؟

- The maximum subarray for any input must be in one of following cases:



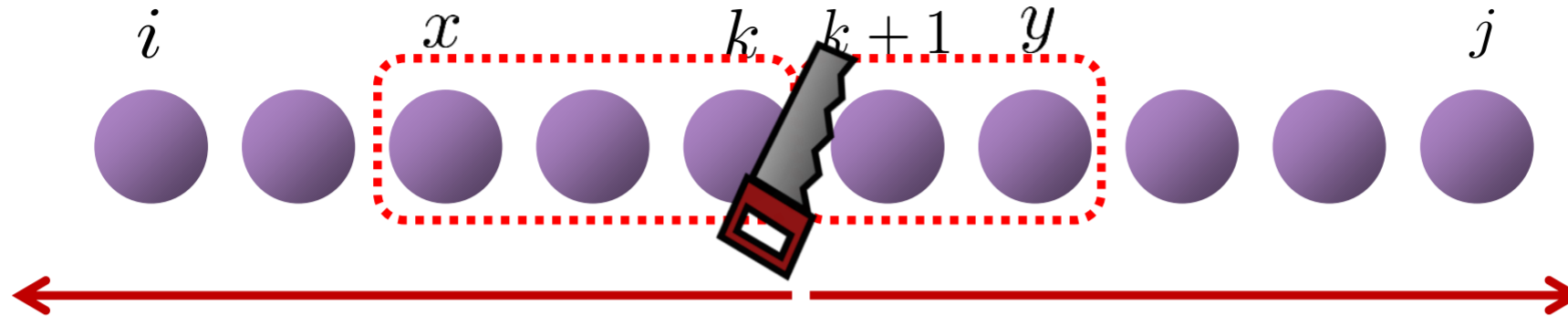
Case 1:  $\text{MaxSub}(A, i, j) = \text{MaxSub}(A, i, k)$

Case 2:  $\text{MaxSub}(A, i, j) = \text{MaxSub}(A, k+1, j)$

Case 3:  $\text{MaxSub}(A, i, j)$  cannot be expressed using  $\text{MaxSub}$ !

# حالت سوم: آرایه ای که از وسط عبور می‌کند!

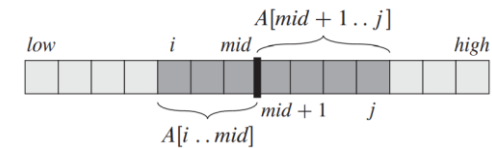
- Goal: find the maximum subarray that crosses the middle



(1) Start from the middle to find the left maximum subarray

(2) Start from the middle to find the right maximum subarray

The solution of Case 3 is the combination of (1) and (2)



- Observation
  - The sum of  $A[x \dots k]$  must be the maximum among  $A[i \dots k]$  (left:  $i \leq k$ )
  - The sum of  $A[k + 1 \dots y]$  must be the maximum among  $A[k + 1 \dots j]$  (right:  $j > k$ )
  - Solvable in linear time  $\rightarrow \Theta(n)$

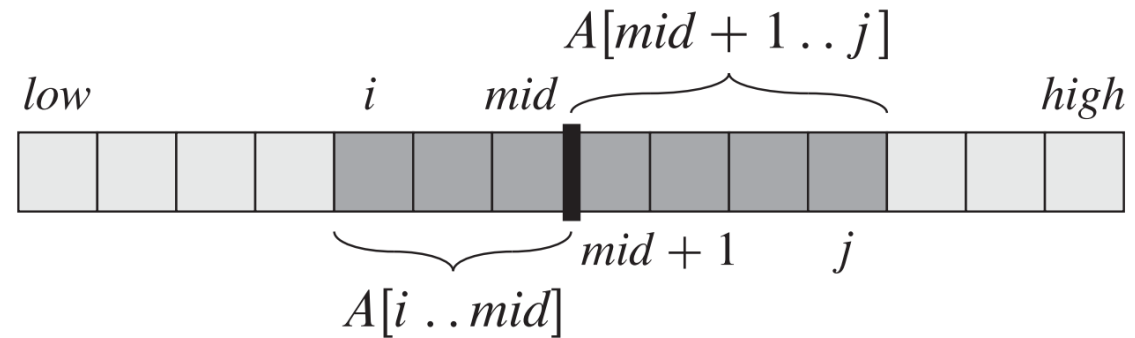
# حل حالت سوم در زمان خطی

FIND-MAX-CROSSING-SUBARRAY( $A, low, mid, high$ )

```

1   $left-sum = -\infty$ 
2   $sum = 0$ 
3  for  $i = mid$  downto  $low$ 
4       $sum = sum + A[i]$ 
5      if  $sum > left-sum$ 
6           $left-sum = sum$ 
7           $max-left = i$ 
8   $right-sum = -\infty$ 
9   $sum = 0$ 
10 for  $j = mid + 1$  to  $high$ 
11      $sum = sum + A[j]$ 
12     if  $sum > right-sum$ 
13          $right-sum = sum$ 
14          $max-right = j$ 
15 return ( $max-left, max-right, left-sum + right-sum$ )

```



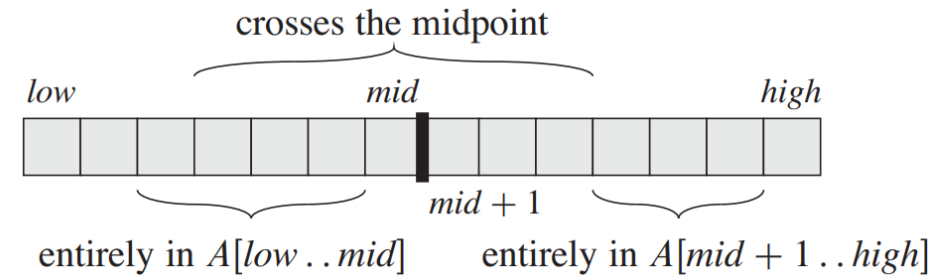
$$\Theta(n)$$

# حل زیرآرایه بیشینه با روش تقسیم و حل

FIND-MAXIMUM-SUBARRAY( $A, low, high$ )

```

1  if  $high == low$ 
2      return ( $low, high, A[low]$ )           // base case: only one element
3  else  $mid = \lfloor (low + high) / 2 \rfloor$ 
4      ( $left-low, left-high, left-sum$ ) =
          FIND-MAXIMUM-SUBARRAY( $A, low, mid$ )
5      ( $right-low, right-high, right-sum$ ) =
          FIND-MAXIMUM-SUBARRAY( $A, mid + 1, high$ )
6      ( $cross-low, cross-high, cross-sum$ ) =
          FIND-MAX-CROSSING-SUBARRAY( $A, low, mid, high$ )
7  if  $left-sum \geq right-sum$  and  $left-sum \geq cross-sum$ 
8      return ( $left-low, left-high, left-sum$ )
9  elseif  $right-sum \geq left-sum$  and  $right-sum \geq cross-sum$ 
10     return ( $right-low, right-high, right-sum$ )
11 else return ( $cross-low, cross-high, cross-sum$ )
    
```



**Initial call:** FIND-MAXIMUM-SUBARRAY( $A, 1, n$ )



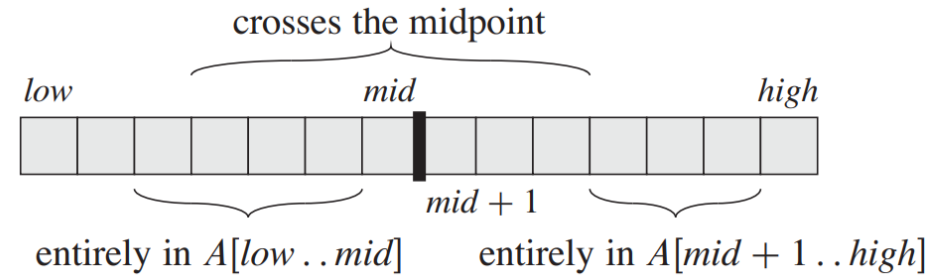
# حل زیرآرایه بیشینه با روش تقسیم و حل

FIND-MAXIMUM-SUBARRAY( $A, low, high$ )

```

1  if  $high == low$ 
2      return ( $low, high, A[low]$ )           // base case: only one element
3  else  $mid = \lfloor (low + high) / 2 \rfloor$ 
Divide ( $left-low, left-high, left-sum$ ) =
        FIND-MAXIMUM-SUBARRAY( $A, low, mid$ )   Conquer
5  ( $right-low, right-high, right-sum$ ) =
        FIND-MAXIMUM-SUBARRAY( $A, mid + 1, high$ )
6  ( $cross-low, cross-high, cross-sum$ ) =
        FIND-MAX-CROSSING-SUBARRAY( $A, low, mid, high$ )
7  if  $left-sum \geq right-sum$  and  $left-sum \geq cross-sum$ 
8      return ( $left-low, left-high, left-sum$ )
9  elseif  $right-sum \geq left-sum$  and  $right-sum \geq cross-sum$ 
10     return ( $right-low, right-high, right-sum$ )
11  else return ( $cross-low, cross-high, cross-sum$ )
    
```

Combine



**Initial call:** FIND-MAXIMUM-SUBARRAY( $A, 1, n$ )

# تحلیل زمانی روش تقسیم و حل MSA

FIND-MAXIMUM-SUBARRAY( $A, low, high$ )

|    |                                                                        |                                       |
|----|------------------------------------------------------------------------|---------------------------------------|
| 1  | <b>if</b> $high == low$                                                |                                       |
| 2  | <b>return</b> ( $low, high, A[low]$ )                                  | // base case: only one element $O(1)$ |
| 3  | <b>else</b> $mid = \lfloor (low + high)/2 \rfloor$                     |                                       |
| 4  | $(left-low, left-high, left-sum) =$                                    |                                       |
|    | FIND-MAXIMUM-SUBARRAY( $A, low, mid$ )                                 | $T(mid - low)$                        |
| 5  | $(right-low, right-high, right-sum) =$                                 |                                       |
|    | FIND-MAXIMUM-SUBARRAY( $A, mid + 1, high$ )                            | $T(high - (mid + 1))$                 |
| 6  | $(cross-low, cross-high, cross-sum) =$                                 |                                       |
|    | FIND-MAX-CROSSING-SUBARRAY( $A, low, mid, high$ )                      | $O(high - mid)$                       |
| 7  | <b>if</b> $left-sum \geq right-sum$ and $left-sum \geq cross-sum$      |                                       |
| 8  | <b>return</b> ( $left-low, left-high, left-sum$ )                      | $O(1)$                                |
| 9  | <b>elseif</b> $right-sum \geq left-sum$ and $right-sum \geq cross-sum$ |                                       |
| 10 | <b>return</b> ( $right-low, right-high, right-sum$ )                   | $O(1)$                                |
| 11 | <b>else return</b> ( $cross-low, cross-high, cross-sum$ )              | $O(1)$                                |

**Initial call:** FIND-MAXIMUM-SUBARRAY( $A, 1, n$ )

# تحلیل زمانی روش تقسیم و حل MSA

## 1. Divide

- Divide a list of size  $n$  into 2 subarrays of size  $n/2$   $\Theta(1)$

## 2. Conquer

- Recursive case ( $n > 1$ )
  - find **MaxSub** for each subarrays  $T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor)$
- Base case ( $n = 1$ )  $\Theta(1)$ 
  - Return itself
- Find **MaxCrossSub** for the original list  $\Theta(n)$

## 3. Combine

- Pick the subarray with the maximum sum among 3 subarrays  $\Theta(1)$

- $T(n)$  = time for running `MaxSubarray(A, i, j)` with  $j - i + 1 = n$

$$T(n) = \begin{cases} O(1) & \text{if } n = 1 \\ T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + O(n) & \text{if } n \geq 2 \end{cases}$$

# تحلیل زمانی روش تقسیم و حل

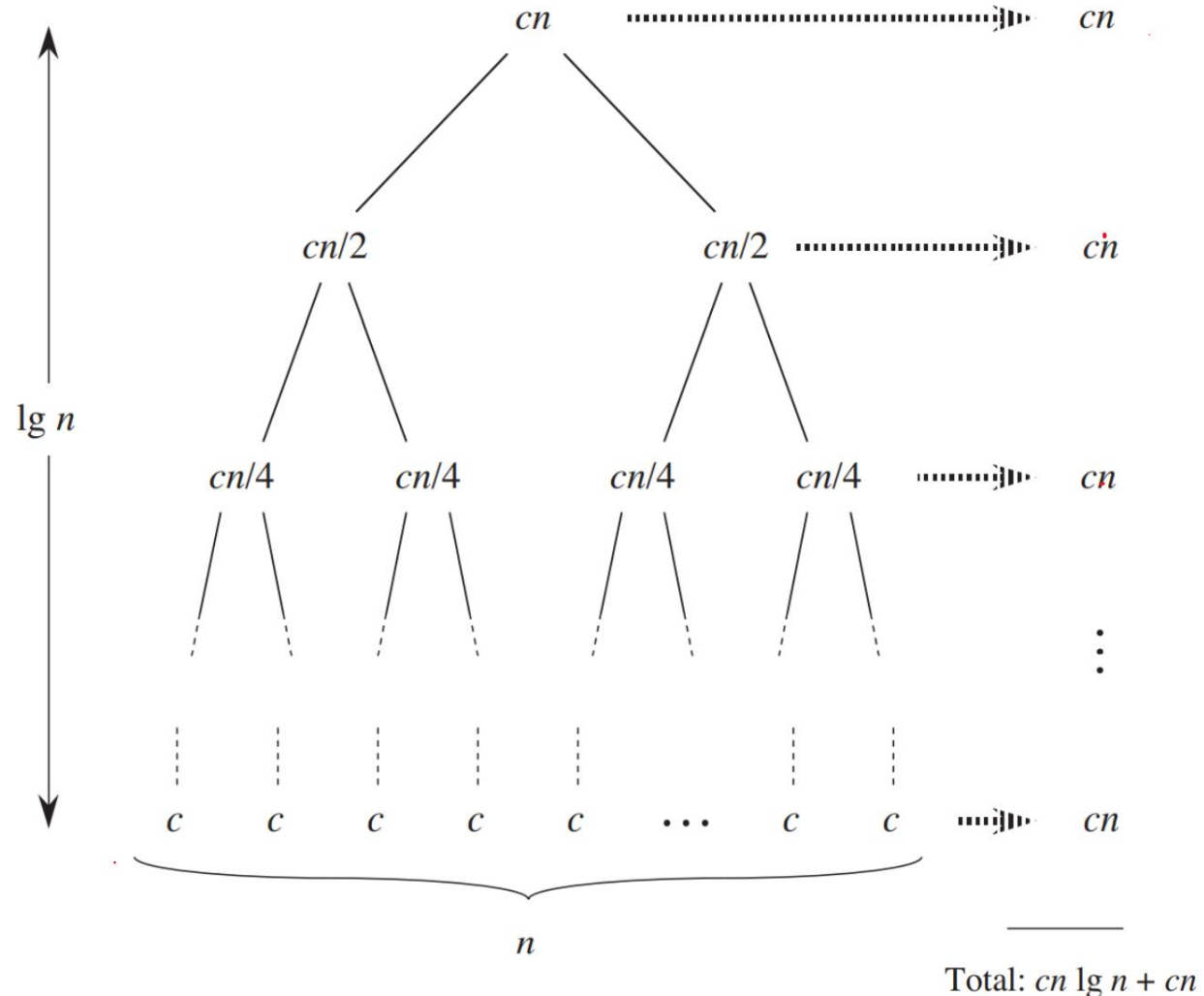
$$T(n) = \begin{cases} O(1) & \text{if } n = 1 \\ T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + O(n) & \text{if } n \geq 2 \end{cases}$$

ساده سازی: تعداد ورودی‌ها توانی از ۲

تعداد ورودی زیرمسئله‌ها  
همیشه عدد صحیح

Merge Sort

$$T(n) = \begin{cases} c & \text{if } n = 1 \\ 2T(n/2) + cn & \text{if } n > 1 \end{cases}$$



آیا این سریع‌ترین راه حل است؟؟ جواب: نه، به تمرین ۵-۴.۱ مراجعه شود

# الگوریتم خطی زیر آرایه بیشینه

## 4.1-5

Use the following ideas to develop a nonrecursive, linear-time algorithm for the maximum-subarray problem. Start at the left end of the array, and progress toward the right, keeping track of the maximum subarray seen so far. Knowing a maximum subarray of  $A[1..j]$ , extend the answer to find a maximum subarray ending at index  $j+1$  by using the following observation: a maximum subarray of  $A[1..j+1]$  is either a maximum subarray of  $A[1..j]$  or a subarray  $A[i..j+1]$ , for some  $1 \leq i \leq j+1$ . Determine a maximum subarray of the form  $A[i..j+1]$  in constant time based on knowing a maximum subarray ending at index  $j$ .



# الگوریتم خطی زیر آرایه بیشینه

MAX-SUBARRAY-LINEAR( $A$ )

$n = A.length$

$max-sum = -\infty$

$ending-here-sum = -\infty$

**for**  $j = 1$  **to**  $n$

$ending-here-high = j$

**if**  $ending-here-sum > 0$

$ending-here-sum = ending-here-sum + A[j]$

**else**  $ending-here-low = j$

$ending-here-sum = A[j]$

**if**  $ending-here-sum > max-sum$

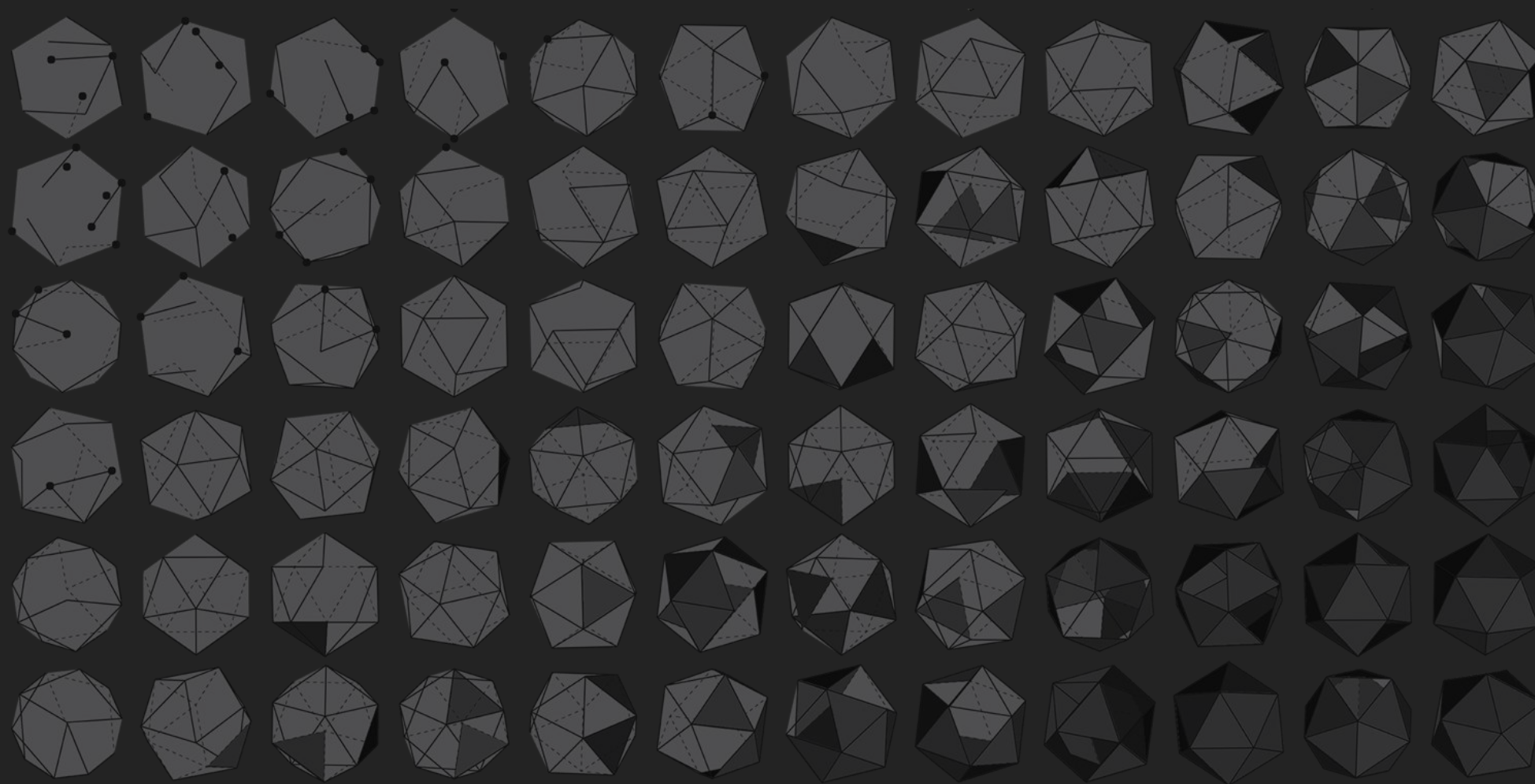
$max-sum = ending-here-sum$

$low = ending-here-low$

$high = ending-here-high$

**return** ( $low, high, max-sum$ )





# روش‌های حل روابط بازگشتی

فصل ۴.۳ الی ۴.۶ کتاب

# مروری بر رابطه بازگشت

Recurrences go hand in hand with the divide-and-conquer paradigm, because they give us a natural way to characterize the running times of divide-and-conquer algorithms.

- A **recurrence** is an equation or inequality that describes a function in terms of its value on smaller inputs.
- Example from MERGE-SORT or MAXIMUM-SUBARRAY

$$T(n) = \begin{cases} \Theta(1) & \text{if } n=1 \\ 2T(n/2) + \Theta(n) & \text{if } n>1 \end{cases}$$

$$T(n) = T(2n/3) + T(n/3) + \Theta(n)$$

$$T(n) = T(n-1) + \Theta(1)$$

Recursive version of linear search

# روش‌های حل رابطه بازگشت

- جایگذاری و استقرای ریاضی
- درخت بازگشت
- قضیه اصلی Master Theorem

$$T(n) = aT(n/b) + f(n)$$

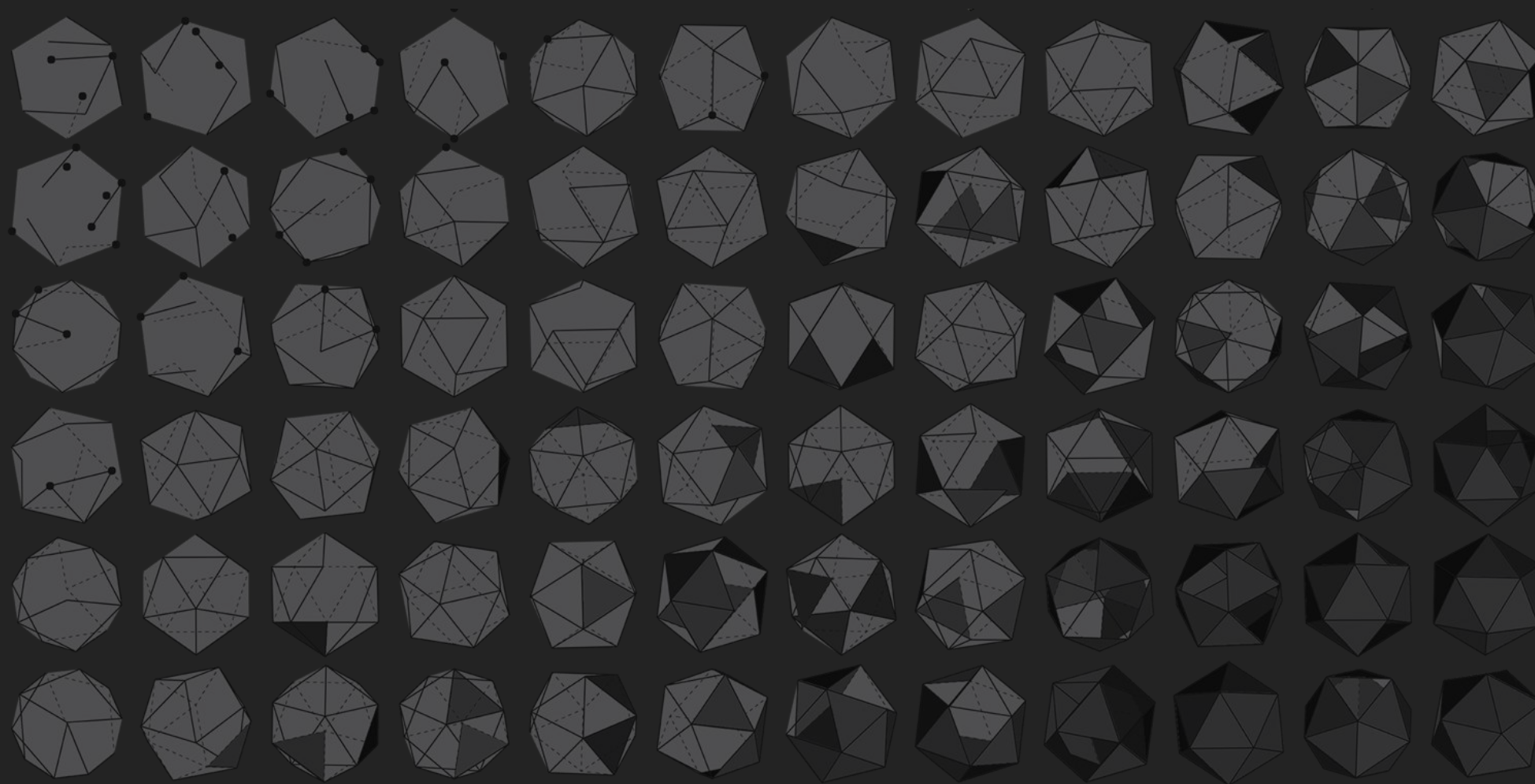
# جزئیات فنی حل رابطه بازگشت

- کف یا جزء صحیح
- سقف
- حالت‌های مرزی

در حل رابطه بازگشتی معمولاً از جزییاتی مانند کف، سقف و حالت‌های مرزی چشم‌پوشی میکنیم زیرا معمولاً تاثیرگذار در نتیجه نیستند!



ولی باید بدانیم در چه شرایطی آنها موثر هستند و باید در نظر گرفته شوند



# روش‌های حل روابط بازگشتی: جایگذاری

فصل ۴.۳ کتاب

# روش جایگذاری

- Involves two steps:
  1. Guess the form of the solution
  2. Use mathematical induction to find the constants and show the solution works
- Drawback: applied only in cases where it is easy to guess at solution

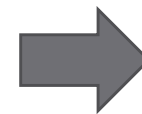
حدس از کجا؟ میتوانیم از روش درخت بازگشت بهره بگیریم

- Example: MERGE-SORT or MAXIMUM-SUBARRAY

$$T(n) = 2T(n/2) + dn$$

- Guess:

$$T(n) = O(n \lg n)$$



Prove by induction:  
 $T(n) \leq cn \lg n$   
 for suitable  $c > 0$ .

$$O(g(n)) = \{ f(n) : \exists c > 0, n_0 > 0 \text{ s.t. } \forall n \geq n_0: f(n) \leq c \cdot g(n) \}$$



# اثبات از طریق استقراء در روش جایگذاری

Guess:

$$T(n) = O(n \lg n)$$

$$T(1) = \Theta(1) \leq cn \lg n$$

• در این مثال پایه استقراء با فرض  $n = 1$  برقرار است؟

$$T(n) \leq cn \lg n \rightarrow T(1) \leq c1 \lg 1 = 0 \quad \times$$

• طبق تعریف ما نیاز به  $n_0$  دلخواه داریم که برای  $n > n_0$  ثابت کنیم که  $T(n) \leq cn \lg n$

• از طرفی برای  $n > 3$  ما مستقیماً وابسته به  $T(1)$  نیستیم. پس بجای  $T(1)$  می‌توانیم  $T(2)$  و  $T(3)$  را به عنوان پایه استقراء جایگزین کنیم و در نظر بگیریم که  $n_0 = 2$

$$T(n) = 2T(n/2) + dn$$

• پایه استقراء  $T(2) = 2T(1) + 2d$  ←  $T(2) \leq 2c \lg 2$  ←  $(2T(1) + 2d)/2 \lg 2 \leq c$

• پایه استقراء  $T(3) = 2T(1) + 3d$  ←  $T(3) \leq 3c \lg 3$  ←  $(2T(1) + 3d)/3 \lg 3 \leq c$

# اثبات از طریق استقراء در روش جایگذاری

- پایه استقراء به شرط  $(T(1)+d) \leq c$  برقرار است
- فرض استقراء: برای  $n < k$  خواهیم داشت:  $T(n) \leq cn \lg n$
- حکم استقراء: برای  $n = k$  نیز نابرابری فوق برقرار خواهد بود

• در مثال اخیر

• مثال رابطه بازگشتی

$$T(n) = 2T(n/2) + dn$$

• حدس

$$T(n) = O(n \lg n)$$

• فرض استقراء:  $T(k/2) \leq c \cdot k/2 \cdot \lg(k/2)$

• حکم استقراء:  $T(k) \leq c \cdot k \cdot \lg(k)$

$$T(k) = 2T(k/2) + dk \leq 2c \cdot k/2 \cdot \lg(k/2) + dk$$

$$\leq c \cdot k \cdot \lg(k/2) + dk$$

$$\leq c \cdot k \cdot [\lg(k) - \lg(2)] + dk$$

$$\leq c \cdot k \cdot \lg(k) - ck + dk$$

$$\longrightarrow dk - ck \leq 0 \longrightarrow$$

$$d \leq c$$

دسته ای از  $c$  که رابطه را صحیح می کند

$$T(k) \leq c \cdot k \cdot \lg(k)$$

# مثال روش جایگذاری

• مثال دیگر...

$$T(n) = 2T(\lfloor n/2 \rfloor) + n$$

حدس:  $T(n) = O(n \lg n)$  باید اثبات شود که  $T(n) \leq cn \lg n$  برای یک  $c > 0$

فرض: حکم قضیه برای  $m = \lfloor n/2 \rfloor$  صادق است. در نتیجه خواهیم داشت:  $T(\lfloor n/2 \rfloor) \leq c \lfloor n/2 \rfloor \lg(\lfloor n/2 \rfloor)$

جایگذاری در رابطه بازگشت

$$T(n) \leq 2(c \lfloor n/2 \rfloor \lg(\lfloor n/2 \rfloor)) + n$$

$$\leq cn \lg(n/2) + n$$

$$= cn \lg n - cn \lg 2 + n$$

$$= cn \lg n - cn + n \longrightarrow -cn + n \leq 0 \longrightarrow c \geq 1$$

$$\leq cn \lg n ,$$

# چگونه حدس خوبی بزنیم؟

- روش عمومی برای یک حدس خوب وجود ندارد!
- تجربه! خلاقیت! شهود! و شاید هم درخت بازگشتی!
- اگر مشابه رابطه بازگشت را قبلاً دیده اید، راه حل مشابه میتواند منطقی باشد!

برای  $n$  های بزرگ  $\lfloor n/2 \rfloor$  و  $\lfloor n/2 \rfloor + 17$  تفاوت چندانی نخواهد داشت!  $\leftarrow T(n) = 2T(\lfloor n/2 \rfloor + 17) + n$

$T(n) = O(n \lg n) \rightarrow$  Exercise 4.3-6

- در برخی موارد مرتبه را صحیح حدس میزنیم ولی استقراء بنوعی درست کار نمیکند!!

$$T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1$$

$$\begin{aligned} T(n) &\leq c \lfloor n/2 \rfloor + c \lceil n/2 \rceil + 1 \\ &= cn + 1, \quad \times \end{aligned}$$

حدس

$$T(n) = O(n) \rightarrow T(n) \leq cn$$

# تصحیح حدس زده شده

- در برخی موارد مرتبه را صحیح حدس میزنیم ولی استقراء بنوعی درست کار نمیکند!!

$$T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1$$

$$\boxed{\text{حدس}} \quad T(n) = O(n) \longrightarrow T(n) \leq cn \quad \times \longrightarrow \boxed{T(n) \leq cn - d, \text{ where } d \geq 0 \text{ is a constant.}}$$

$$\begin{aligned} T(n) &\leq (c \lfloor n/2 \rfloor - d) + (c \lceil n/2 \rceil - d) + 1 \\ &= cn - 2d + 1 \\ &\leq cn - d, \end{aligned}$$

$$\boxed{d \geq 1}$$

۱. درجه جمله اضافه کمتر از حکم باشد: از حکم یک جمله با درجه کمتر کم میکنیم
۲. درجه جمله اضافه با حکم برابر باشد: یک فاکتور لگاریتم در حکم کمتر حدس زده ایم
۳. درجه جمله اضافه بیشتر از حکم باشد: باید حکم از درجه بالاتری باشد

# تصحیح حدس زده شده

۱. درجه جمله اضافه کمتر از حکم باشد: به حکم یک جمله از درجه کمتر اضافه میکنیم
۲. درجه جمله اضافه با حکم برابر باشد: یک فاکتور لگاریتم در حکم کمتر حدس زدیم
۳. درجه جمله اضافه بیشتر از حکم باشد: باید حکم از درجه بالاتری باشد

$$T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + n$$

$$\boxed{\text{حدس}} \quad T(n) = O(n) \longrightarrow T(n) \leq cn$$

$$\begin{aligned} T(n) &\leq c \lfloor n/2 \rfloor + c \lceil n/2 \rceil + n \\ &= cn + n, \end{aligned}$$

حالت دوم، درجه جمله اضافی برابر حکم است. پس یک لگاریتم کم حدس زده ایم!



# تغییر متغیر برای حل استقرای

- در برخی موارد یک تغییر متغیر ساده شما را به یک رابطه بازگشتی آشنا میرساند

$$\begin{array}{lcl}
 \boxed{T(n) = 2T(\sqrt{n}) + \lg n} & \left. \begin{array}{l} \\ \\ \\ \end{array} \right\} & \begin{array}{l} T(2^m) = 2T(2^{m/2}) + m \\ \\ S(m) = T(2^m) \end{array} \\
 m = \lg n & & \left. \begin{array}{l} \\ \\ \end{array} \right\} S(m) = 2S(m/2) + m \\
 & & \downarrow \\
 & & S(m) = O(m \lg m) \\
 & & \downarrow \\
 \boxed{T(n) = T(2^m) = S(m) = O(m \lg m) = O(\lg n \lg \lg n)}
 \end{array}$$

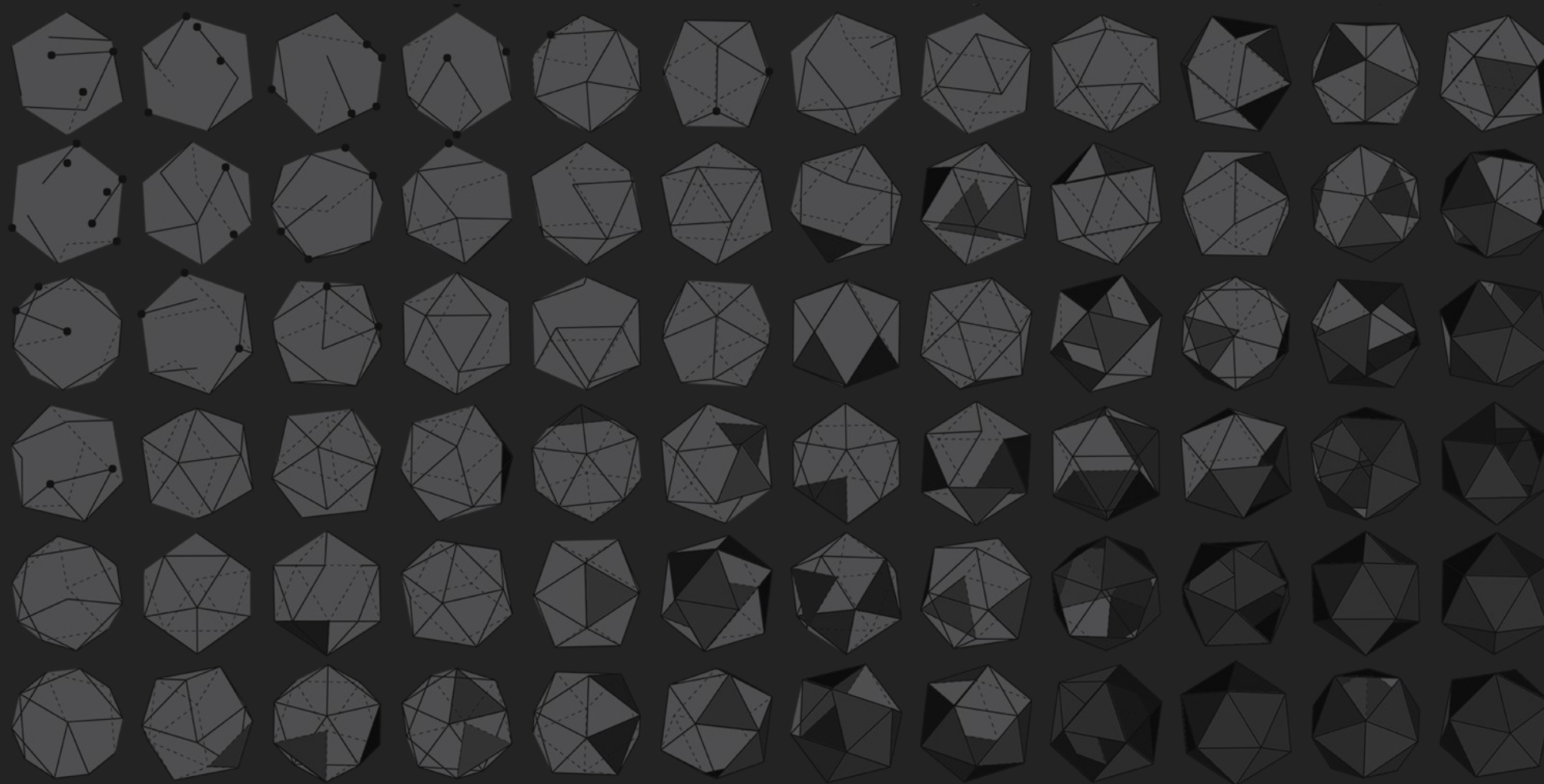
# خطاهای احتمالی در استقراء

• خطا کردن با عدم دقت در استقراء خیلی هم سخت نیست!

$$T(n) = 2T(\lfloor n/2 \rfloor) + n$$

حدس  $T(n) = O(n) \longrightarrow T(n) \leq cn \quad \times$

$$\begin{aligned} T(n) &\leq 2(c \lfloor n/2 \rfloor) + n \\ &\leq cn + n \\ &= O(n), \quad \Longleftarrow \text{wrong!!} \end{aligned}$$



# روش‌های حل رابطه بازگشتی: قضیه اصلی Master Theorem

فصل ۴.۵ کتاب

# قضیه اصلی Master Theorem

$$T(n) = aT(n/b) + f(n) \quad a \geq 1 \text{ and } b > 1$$

رابطه بازگشت  $T(n)$  مسئله را به  $a$  زیرمسئله مساوی به اندازه  $n/b$  تقسیم میکند و هزینه تقسیم و ترکیب زیرمسئله‌ها برابر  $f(n)$  است

$n/b$  to mean either  $\lfloor n/b \rfloor$  or  $\lceil n/b \rceil$

1. If  $f(n) = O(n^{\log_b a - \epsilon})$  for some constant  $\epsilon > 0$ , then  $T(n) = \Theta(n^{\log_b a})$ .
2. If  $f(n) = \Theta(n^{\log_b a})$ , then  $T(n) = \Theta(n^{\log_b a} \lg n) = \Theta(f(n) \lg n)$
3. If  $f(n) = \Omega(n^{\log_b a + \epsilon})$  for some constant  $\epsilon > 0$ , and if  $af(n/b) \leq cf(n)$  for some constant  $c < 1$  and all sufficiently large  $n$ , then  $T(n) = \Theta(f(n))$ . شرط منظم بودن

$$f(n) \square n^{\log_b a}$$

# مثال قضیه اصلی - ۱

$$T(n) = 9T(n/3) + n$$

$$f(n) \square n^{\log_b a}$$

$$T(n) = aT(n/b) + f(n) \quad a \geq 1 \text{ and } b > 1$$

1. If  $f(n) = O(n^{\log_b a - \epsilon})$  for some constant  $\epsilon > 0$ , then  $T(n) = \Theta(n^{\log_b a})$ .
2. If  $f(n) = \Theta(n^{\log_b a})$ , then  $T(n) = \Theta(n^{\log_b a} \lg n) = \Theta(f(n) \lg n)$
3. If  $f(n) = \Omega(n^{\log_b a + \epsilon})$  for some constant  $\epsilon > 0$ , and if  $af(n/b) \leq cf(n)$  for some constant  $c < 1$  and all sufficiently large  $n$ , then  $T(n) = \Theta(f(n))$ . ■

$$n^{\log_b a} = n^{\log_3 9} = \Theta(n^2)$$

$$f(n) = O(n^{\log_3 9 - \epsilon}), \text{ where } \epsilon = 1$$

$$T(n) = \Theta(n^2)$$

## مثال قضیه اصلی - ۲

$$T(n) = T(2n/3) + 1$$

$$f(n) \square n^{\log_b a}$$

$$T(n) = aT(n/b) + f(n) \quad a \geq 1 \text{ and } b > 1$$

1. If  $f(n) = O(n^{\log_b a - \epsilon})$  for some constant  $\epsilon > 0$ , then  $T(n) = \Theta(n^{\log_b a})$ .
2. If  $f(n) = \Theta(n^{\log_b a})$ , then  $T(n) = \Theta(n^{\log_b a} \lg n) = \Theta(f(n) \lg n)$
3. If  $f(n) = \Omega(n^{\log_b a + \epsilon})$  for some constant  $\epsilon > 0$ , and if  $af(n/b) \leq cf(n)$  for some constant  $c < 1$  and all sufficiently large  $n$ , then  $T(n) = \Theta(f(n))$ . ■

$$n^{\log_b a} = n^{\log_{3/2} 1} = n^0 = 1$$

$$f(n) = \Theta(n^{\log_b a}) = \Theta(1)$$

$$T(n) = \Theta(\lg n)$$

## مثال قضیه اصلی - ۳

$$T(n) = 3T(n/4) + n \lg n$$

$$f(n) \square n^{\log_b a}$$

$$T(n) = aT(n/b) + f(n) \quad a \geq 1 \text{ and } b > 1$$

1. If  $f(n) = O(n^{\log_b a - \epsilon})$  for some constant  $\epsilon > 0$ , then  $T(n) = \Theta(n^{\log_b a})$ .
2. If  $f(n) = \Theta(n^{\log_b a})$ , then  $T(n) = \Theta(n^{\log_b a} \lg n) = \Theta(f(n) \lg n)$
3. If  $f(n) = \Omega(n^{\log_b a + \epsilon})$  for some constant  $\epsilon > 0$ , and if  $af(n/b) \leq cf(n)$  for some constant  $c < 1$  and all sufficiently large  $n$ , then  $T(n) = \Theta(f(n))$ . ■

$$n^{\log_b a} = n^{\log_4 3} = O(n^{0.793})$$

$$f(n) = \Omega(n^{\log_4 3 + \epsilon}), \text{ where } \epsilon \approx 0.2$$

$$af(n/b) = 3(n/4) \lg(n/4) \leq (3/4)n \lg n = cf(n) \text{ for } c = 3/4$$

$$T(n) = \Theta(n \lg n)$$



## مثال قضیه اصلی - ۴

$$T(n) = 2T(n/2) + n \lg n$$

$$n^{\log_b a} = n \quad \square \quad f(n) = n \lg n$$

X

is not *polynomially* larger

The ratio  $f(n)/n^{\log_b a} = (n \lg n)/n = \lg n$  is asymptotically less than  $n^\epsilon$  for any positive constant  $\epsilon$

the recurrence falls into the gap between case 2 and case 3

Show that if  $f(n) = \Theta(n^{\log_b a} \lg^k n)$ , where  $k \geq 0$ , then the master recurrence has solution  $T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$ . For simplicity, confine your analysis to exact powers of  $b$ .

Exercise 4.6-2

$$T(n) = aT(n/b) + f(n) \quad a \geq 1 \text{ and } b > 1$$

1. If  $f(n) = O(n^{\log_b a - \epsilon})$  for some constant  $\epsilon > 0$ , then  $T(n) = \Theta(n^{\log_b a})$ .
2. If  $f(n) = \Theta(n^{\log_b a})$ , then  $T(n) = \Theta(n^{\log_b a} \lg n) = \Theta(f(n) \lg n)$
3. If  $f(n) = \Omega(n^{\log_b a + \epsilon})$  for some constant  $\epsilon > 0$ , and if  $a f(n/b) \leq c f(n)$  for some constant  $c < 1$  and all sufficiently large  $n$ , then  $T(n) = \Theta(f(n))$ . ■

# مثال قضیه اصلی - ۵

$$T(n) = 2T(n/2) + \Theta(n)$$

مرتبه زمانی الگوریتم‌های تقسیم و حل Mergesort و MSA

$$T(n) = aT(n/b) + f(n) \quad a \geq 1 \text{ and } b > 1$$

1. If  $f(n) = O(n^{\log_b a - \epsilon})$  for some constant  $\epsilon > 0$ , then  $T(n) = \Theta(n^{\log_b a})$ .
2. If  $f(n) = \Theta(n^{\log_b a})$ , then  $T(n) = \Theta(n^{\log_b a} \lg n) = \Theta(f(n) \lg n)$
3. If  $f(n) = \Omega(n^{\log_b a + \epsilon})$  for some constant  $\epsilon > 0$ , and if  $af(n/b) \leq cf(n)$  for some constant  $c < 1$  and all sufficiently large  $n$ , then  $T(n) = \Theta(f(n))$ . ■

$$n^{\log_b a} = n^{\log_2 2} = n$$

$$f(n) = \Theta(n)$$

$$T(n) = \Theta(n \lg n)$$

## مثال قضیه اصلی - ۶

$$T(n) = 2T(n/4) + \sqrt{n}$$

$$f(n) \square n^{\log_b a}$$

$$T(n) = aT(n/b) + f(n) \quad a \geq 1 \text{ and } b > 1$$

1. If  $f(n) = O(n^{\log_b a - \epsilon})$  for some constant  $\epsilon > 0$ , then  $T(n) = \Theta(n^{\log_b a})$ .
2. If  $f(n) = \Theta(n^{\log_b a})$ , then  $T(n) = \Theta(n^{\log_b a} \lg n) = \Theta(f(n) \lg n)$
3. If  $f(n) = \Omega(n^{\log_b a + \epsilon})$  for some constant  $\epsilon > 0$ , and if  $af(n/b) \leq cf(n)$  for some constant  $c < 1$  and all sufficiently large  $n$ , then  $T(n) = \Theta(f(n))$ . ■

$$n^{\log_b a} = n^{\log_4 2} = \sqrt{n}$$

$$\sqrt{n} = \Theta(n^{\log_4 2}) \quad \boxed{\text{case 2}}$$

$$T(n) = \Theta(\sqrt{n} \lg n)$$

# مثال قضیه اصلی - ۷

$$T(n) = T(\sqrt{n}) + 1$$

استفاده از تغییر متغیر

$$T(n) = aT(n/b) + f(n) \quad a \geq 1 \text{ and } b > 1$$

1. If  $f(n) = O(n^{\log_b a - \epsilon})$  for some constant  $\epsilon > 0$ , then  $T(n) = \Theta(n^{\log_b a})$ .
2. If  $f(n) = \Theta(n^{\log_b a})$ , then  $T(n) = \Theta(n^{\log_b a} \lg n) = \Theta(f(n) \lg n)$
3. If  $f(n) = \Omega(n^{\log_b a + \epsilon})$  for some constant  $\epsilon > 0$ , and if  $a f(n/b) \leq c f(n)$  for some constant  $c < 1$  and all sufficiently large  $n$ , then  $T(n) = \Theta(f(n))$ . ■

$$m = \lg n$$

$$T(2^m) = T(2^{m/2}) + 1$$

$$S(m) = T(2^m)$$

$$S(m) = S(m/2) + 1$$

$$n^{\log_b a} = n^{\log_2 1} = n^0 = 1 \quad f(n) = 1$$

$$f(n) = \Theta(1)$$

case 2 applies and  $S(m) = \Theta(\lg m)$

$$T(n) = \Theta(\lg \lg n)$$

# درک قضیه اصلی

Under the simplifying assumption that  $T(n)$  is defined only on exact powers of  $b > 1$  that is, for  $n = 1, b, b^2, \dots$

$$T(n) = aT(n/b) + f(n) \quad a \geq 1 \text{ and } b > 1$$

1. If  $f(n) = O(n^{\log_b a - \epsilon})$  for some constant  $\epsilon > 0$ , then  $T(n) = \Theta(n^{\log_b a})$ .
2. If  $f(n) = \Theta(n^{\log_b a})$ , then  $T(n) = \Theta(n^{\log_b a} \lg n) = \Theta(f(n) \lg n)$
3. If  $f(n) = \Omega(n^{\log_b a + \epsilon})$  for some constant  $\epsilon > 0$ , and if  $af(n/b) \leq cf(n)$  for some constant  $c < 1$  and all sufficiently large  $n$ , then  $T(n) = \Theta(f(n))$ . ■

## Lemma 4.2

Let  $a \geq 1$  and  $b > 1$  be constants, and let  $f(n)$  be a nonnegative function defined on exact powers of  $b$ . Define  $T(n)$  on exact powers of  $b$  by the recurrence

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1, \\ aT(n/b) + f(n) & \text{if } n = b^i, \end{cases}$$

where  $i$  is a positive integer. Then

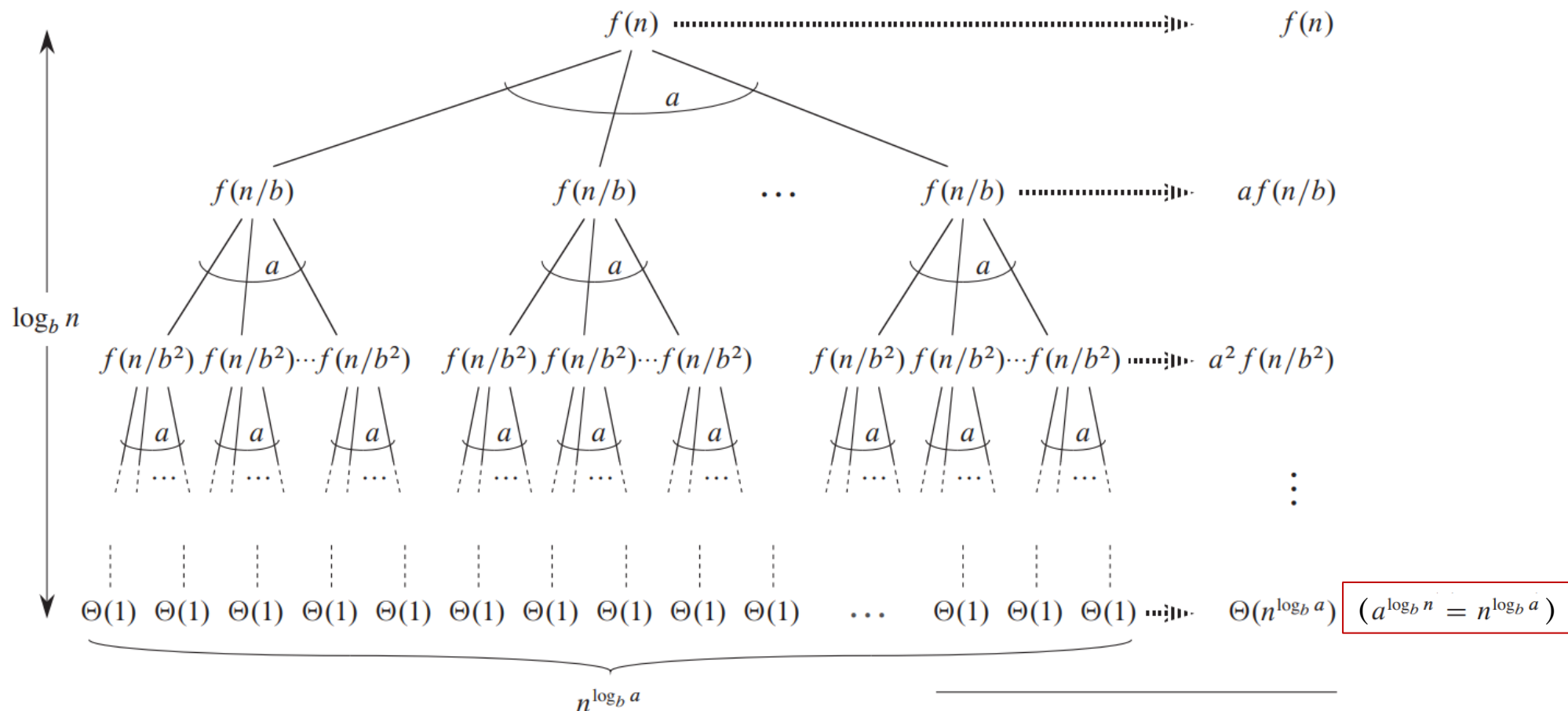
$$T(n) = \Theta(n^{\log_b a}) + \sum_{j=0}^{\log_b n - 1} a^j f(n/b^j). \quad (4.21)$$

# درک قضیه اصلی

$$T(n) = aT(n/b) + f(n) \quad a \geq 1 \text{ and } b > 1$$

1. If  $f(n) = O(n^{\log_b a - \epsilon})$  for some constant  $\epsilon > 0$ , then  $T(n) = \Theta(n^{\log_b a})$ .
2. If  $f(n) = \Theta(n^{\log_b a})$ , then  $T(n) = \Theta(n^{\log_b a} \lg n) = \Theta(f(n) \lg n)$
3. If  $f(n) = \Omega(n^{\log_b a + \epsilon})$  for some constant  $\epsilon > 0$ , and if  $af(n/b) \leq cf(n)$  for some constant  $c < 1$  and all sufficiently large  $n$ , then  $T(n) = \Theta(f(n))$ . ■

Under the simplifying assumption that  $T(n)$  is defined only on exact powers of  $b > 1$  that is, for  $n = 1, b, b^2, \dots$



$$\text{Total: } \Theta(n^{\log_b a}) + \sum_{j=0}^{\log_b n - 1} a^j f(n/b^j)$$

# درک قضیه اصلی

$$T(n) = aT(n/b) + f(n) \quad a \geq 1 \text{ and } b > 1$$

1. If  $f(n) = O(n^{\log_b a - \epsilon})$  for some constant  $\epsilon > 0$ , then  $T(n) = \Theta(n^{\log_b a})$ .
2. If  $f(n) = \Theta(n^{\log_b a})$ , then  $T(n) = \Theta(n^{\log_b a} \lg n) = \Theta(f(n) \lg n)$
3. If  $f(n) = \Omega(n^{\log_b a + \epsilon})$  for some constant  $\epsilon > 0$ , and if  $af(n/b) \leq cf(n)$  for some constant  $c < 1$  and all sufficiently large  $n$ , then  $T(n) = \Theta(f(n))$ . ■

$$T(n) = \Theta(n^{\log_b a}) + \sum_{j=0}^{\log_b n - 1} a^j f(n/b^j)$$



# درک قضیه اصلی

Under the simplifying assumption that  $T(n)$  is defined only on exact powers of  $b > 1$  that is, for  $n = 1, b, b^2, \dots$

$$T(n) = aT(n/b) + f(n) \quad a \geq 1 \text{ and } b > 1$$

1. If  $f(n) = O(n^{\log_b a - \epsilon})$  for some constant  $\epsilon > 0$ , then  $T(n) = \Theta(n^{\log_b a})$ .
2. If  $f(n) = \Theta(n^{\log_b a})$ , then  $T(n) = \Theta(n^{\log_b a} \lg n) = \Theta(f(n) \lg n)$
3. If  $f(n) = \Omega(n^{\log_b a + \epsilon})$  for some constant  $\epsilon > 0$ , and if  $af(n/b) \leq cf(n)$  for some constant  $c < 1$  and all sufficiently large  $n$ , then  $T(n) = \Theta(f(n))$ . ■

## Lemma 4.3

Let  $a \geq 1$  and  $b > 1$  be constants, and let  $f(n)$  be a nonnegative function defined on exact powers of  $b$ . A function  $g(n)$  defined over exact powers of  $b$  by

$$g(n) = \sum_{j=0}^{\log_b n - 1} a^j f(n/b^j) \quad (4.22)$$

has the following asymptotic bounds for exact powers of  $b$ :

1. If  $f(n) = O(n^{\log_b a - \epsilon})$  for some constant  $\epsilon > 0$ , then  $g(n) = O(n^{\log_b a})$ .
2. If  $f(n) = \Theta(n^{\log_b a})$ , then  $g(n) = \Theta(n^{\log_b a} \lg n)$ .
3. If  $af(n/b) \leq cf(n)$  for some constant  $c < 1$  and for all sufficiently large  $n$ , then  $g(n) = \Theta(f(n))$ .

# درک قضیه اصلی

**Proof** For case 1, we have  $f(n) = O(n^{\log_b a - \epsilon})$ , which implies that  $f(n/b^j) = O((n/b^j)^{\log_b a - \epsilon})$ . Substituting into equation (4.22) yields

$$g(n) = O\left(\sum_{j=0}^{\log_b n - 1} a^j \left(\frac{n}{b^j}\right)^{\log_b a - \epsilon}\right). \quad (4.23)$$

We bound the summation within the  $O$ -notation by factoring out terms and simplifying, which leaves an increasing geometric series:

$$\begin{aligned} \sum_{j=0}^{\log_b n - 1} a^j \left(\frac{n}{b^j}\right)^{\log_b a - \epsilon} &= n^{\log_b a - \epsilon} \sum_{j=0}^{\log_b n - 1} \left(\frac{ab^\epsilon}{b^{\log_b a}}\right)^j \\ &= n^{\log_b a - \epsilon} \sum_{j=0}^{\log_b n - 1} (b^\epsilon)^j \\ &= n^{\log_b a - \epsilon} \left(\frac{b^{\epsilon \log_b n} - 1}{b^\epsilon - 1}\right) \\ &= n^{\log_b a - \epsilon} \left(\frac{n^\epsilon - 1}{b^\epsilon - 1}\right). \end{aligned}$$

$$\sum_{k=0}^n x^k = \frac{x^{n+1} - 1}{x - 1}$$

Since  $b$  and  $\epsilon$  are constants, we can rewrite the last expression as  $n^{\log_b a - \epsilon} O(n^\epsilon) = O(n^{\log_b a})$ . Substituting this expression for the summation in equation (4.23) yields

$$g(n) = O(n^{\log_b a}),$$

thereby proving case 1.

## Lemma 4.3

Let  $a \geq 1$  and  $b > 1$  be constants, and let  $f(n)$  be a nonnegative function defined on exact powers of  $b$ . A function  $g(n)$  defined over exact powers of  $b$  by

$$g(n) = \sum_{j=0}^{\log_b n - 1} a^j f(n/b^j) \quad (4.22)$$

has the following asymptotic bounds for exact powers of  $b$ :

1. If  $f(n) = O(n^{\log_b a - \epsilon})$  for some constant  $\epsilon > 0$ , then  $g(n) = O(n^{\log_b a})$ .
2. If  $f(n) = \Theta(n^{\log_b a})$ , then  $g(n) = \Theta(n^{\log_b a} \lg n)$ .
3. If  $af(n/b) \leq cf(n)$  for some constant  $c < 1$  and for all sufficiently large  $n$ , then  $g(n) = \Theta(f(n))$ .

# درک قضیه اصلی

Because case 2 assumes that  $f(n) = \Theta(n^{\log_b a})$ , we have that  $f(n/b^j) = \Theta((n/b^j)^{\log_b a})$ . Substituting into equation (4.22) yields

$$g(n) = \Theta \left( \sum_{j=0}^{\log_b n - 1} a^j \left( \frac{n}{b^j} \right)^{\log_b a} \right). \quad (4.24)$$

We bound the summation within the  $\Theta$ -notation as in case 1, but this time we do not obtain a geometric series. Instead, we discover that every term of the summation is the same:

$$\begin{aligned} \sum_{j=0}^{\log_b n - 1} a^j \left( \frac{n}{b^j} \right)^{\log_b a} &= n^{\log_b a} \sum_{j=0}^{\log_b n - 1} \left( \frac{a}{b^{\log_b a}} \right)^j \\ &= n^{\log_b a} \sum_{j=0}^{\log_b n - 1} 1 \\ &= n^{\log_b a} \log_b n. \end{aligned}$$

Substituting this expression for the summation in equation (4.24) yields

$$\begin{aligned} g(n) &= \Theta(n^{\log_b a} \log_b n) \\ &= \Theta(n^{\log_b a} \lg n), \end{aligned}$$

proving case 2.

## Lemma 4.3

Let  $a \geq 1$  and  $b > 1$  be constants, and let  $f(n)$  be a nonnegative function defined on exact powers of  $b$ . A function  $g(n)$  defined over exact powers of  $b$  by

$$g(n) = \sum_{j=0}^{\log_b n - 1} a^j f(n/b^j) \quad (4.22)$$

has the following asymptotic bounds for exact powers of  $b$ :

1. If  $f(n) = O(n^{\log_b a - \epsilon})$  for some constant  $\epsilon > 0$ , then  $g(n) = O(n^{\log_b a})$ .
2. If  $f(n) = \Theta(n^{\log_b a})$ , then  $g(n) = \Theta(n^{\log_b a} \lg n)$ .
3. If  $af(n/b) \leq cf(n)$  for some constant  $c < 1$  and for all sufficiently large  $n$ , then  $g(n) = \Theta(f(n))$ .

# درک قضیه اصلی

We prove case 3 similarly. Since  $f(n)$  appears in the definition (4.22) of  $g(n)$  and all terms of  $g(n)$  are nonnegative, we can conclude that  $g(n) = \Omega(f(n))$  for exact powers of  $b$ . We assume in the statement of the lemma that  $af(n/b) \leq cf(n)$  for some constant  $c < 1$  and all sufficiently large  $n$ . We rewrite this assumption as  $f(n/b) \leq (c/a)f(n)$  and iterate  $j$  times, yielding  $f(n/b^j) \leq (c/a)^j f(n)$  or, equivalently,  $a^j f(n/b^j) \leq c^j f(n)$ , where we assume that the values we iterate on are sufficiently large. Since the last, and smallest, such value is  $n/b^{j-1}$ , it is enough to assume that  $n/b^{j-1}$  is sufficiently large.

Substituting into equation (4.22) and simplifying yields a geometric series, but unlike the series in case 1, this one has decreasing terms. We use an  $O(1)$  term to capture the terms that are not covered by our assumption that  $n$  is sufficiently large:

$$\begin{aligned} g(n) &= \sum_{j=0}^{\log_b n - 1} a^j f(n/b^j) \\ &\leq \sum_{j=0}^{\log_b n - 1} c^j f(n) + O(1) \\ &\leq f(n) \sum_{j=0}^{\infty} c^j + O(1) \\ &= f(n) \left( \frac{1}{1-c} \right) + O(1) \\ &= O(f(n)), \end{aligned}$$

$$\sum_{k=0}^{\infty} x^k = \frac{1}{1-x}$$

since  $c$  is a constant. Thus, we can conclude that  $g(n) = \Theta(f(n))$  for exact powers of  $b$ . With case 3 proved, the proof of the lemma is complete. ■

## Lemma 4.3

Let  $a \geq 1$  and  $b > 1$  be constants, and let  $f(n)$  be a nonnegative function defined on exact powers of  $b$ . A function  $g(n)$  defined over exact powers of  $b$  by

$$g(n) = \sum_{j=0}^{\log_b n - 1} a^j f(n/b^j) \quad (4.22)$$

has the following asymptotic bounds for exact powers of  $b$ :

1. If  $f(n) = O(n^{\log_b a - \epsilon})$  for some constant  $\epsilon > 0$ , then  $g(n) = O(n^{\log_b a})$ .
2. If  $f(n) = \Theta(n^{\log_b a})$ , then  $g(n) = \Theta(n^{\log_b a} \lg n)$ .
3. If  $af(n/b) \leq cf(n)$  for some constant  $c < 1$  and for all sufficiently large  $n$ , then  $g(n) = \Theta(f(n))$ .

## ***Lemma 4.4***

Let  $a \geq 1$  and  $b > 1$  be constants, and let  $f(n)$  be a nonnegative function defined on exact powers of  $b$ . Define  $T(n)$  on exact powers of  $b$  by the recurrence

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1, \\ aT(n/b) + f(n) & \text{if } n = b^i, \end{cases}$$

where  $i$  is a positive integer. Then  $T(n)$  has the following asymptotic bounds for exact powers of  $b$ :

1. If  $f(n) = O(n^{\log_b a - \epsilon})$  for some constant  $\epsilon > 0$ , then  $T(n) = \Theta(n^{\log_b a})$ .
2. If  $f(n) = \Theta(n^{\log_b a})$ , then  $T(n) = \Theta(n^{\log_b a} \lg n)$ .
3. If  $f(n) = \Omega(n^{\log_b a + \epsilon})$  for some constant  $\epsilon > 0$ , and if  $af(n/b) \leq cf(n)$  for some constant  $c < 1$  and all sufficiently large  $n$ , then  $T(n) = \Theta(f(n))$ .

# درک قضیه اصلی

## Exercise 4.6-2

Show that if  $f(n) = \Theta(n^{\log_b a} \lg^k n)$ , where  $k \geq 0$ , then the master recurrence has solution  $T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$ . For simplicity, confine your analysis to exact powers of  $b$ .

$$\begin{aligned}
 T(n) = aT(n/b) + f(n) &\longrightarrow T(n) = \Theta(n^{\log_b a}) + \sum_{j=0}^{\log_b n - 1} a^j f(n/b^j) \\
 f(n) = \Theta(n^{\log_b a} \lg^k n) &\longrightarrow T(n) = \Theta(n^{\log_b a}) + \sum_{j=0}^{\log_b n - 1} a^j \Theta\left((n/b^j)^{\log_b a} \lg^k(n/b^j)\right) \\
 &= \Theta(n^{\log_b a}) + \Theta\left(\sum_{j=0}^{\log_b n - 1} a^j (n/b^j)^{\log_b a} \lg^k(n/b^j)\right) \\
 &= \Theta(n^{\log_b a}) + \Theta\left(n^{\log_b a} \sum_{j=0}^{\log_b n - 1} \left(\frac{a}{b^{\log_b a}}\right)^j \lg^k(n/b^j)\right) \\
 &= \Theta(n^{\log_b a}) + \Theta\left(n^{\log_b a} \sum_{j=0}^{\log_b n - 1} \lg^k(n/b^j)\right)
 \end{aligned}$$



# درک قضیه اصلی

$$= \Theta(n^{\log_b a}) + \Theta\left(n^{\log_b a} \sum_{j=0}^{\log_b n - 1} \lg^k(n/b^j)\right)$$

$$\begin{aligned} \lg^k(n/b^j) &= (\lg n - \lg b^j)^k \\ &= \lg^k n - \lg^k b^j \rightarrow o(\lg^k n) \end{aligned}$$

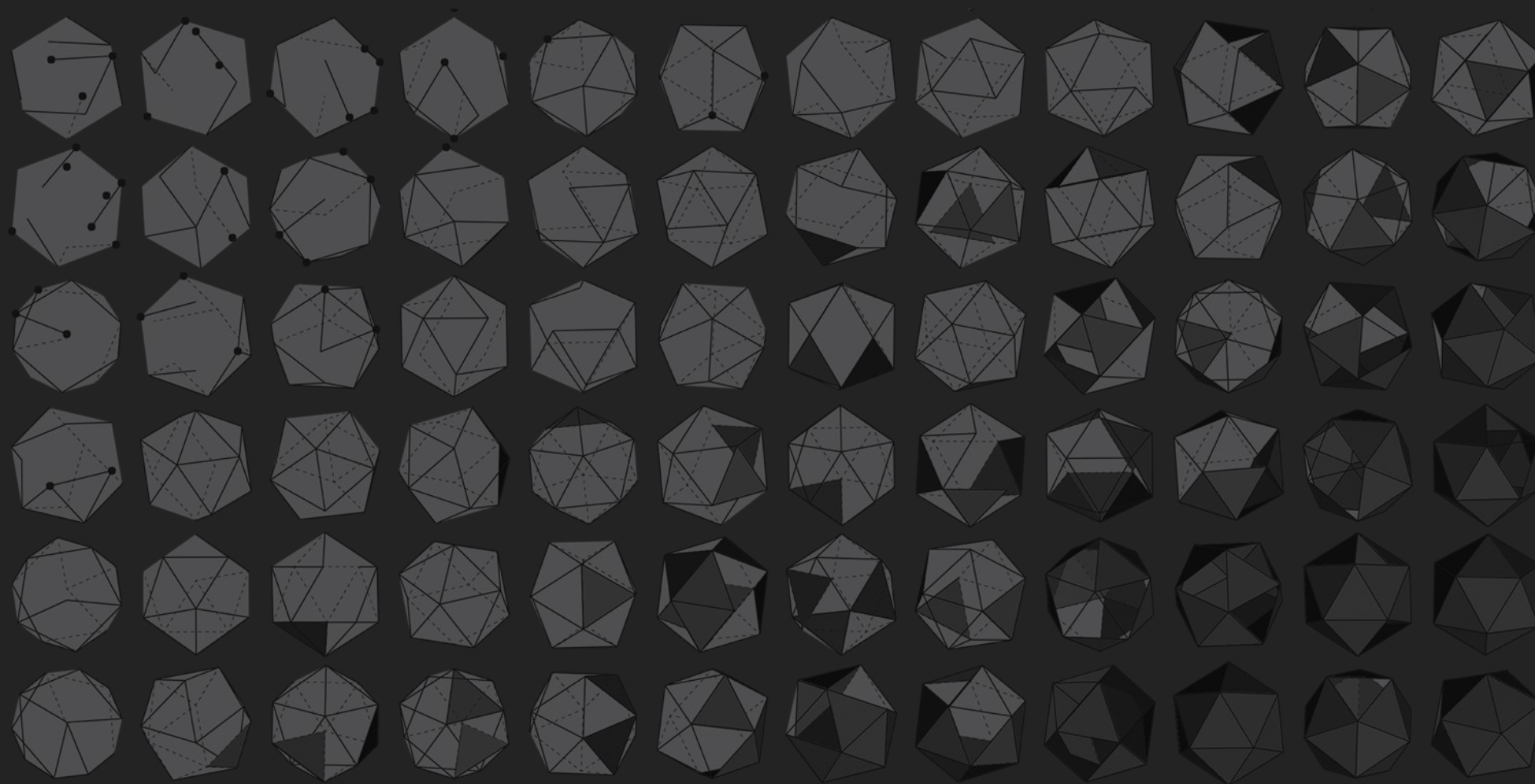
$$\begin{aligned} T(n) &= \Theta(n^{\log_b a}) + \Theta\left(n^{\log_b a} \sum_{j=0}^{\log_b n - 1} \lg^k(n/b^j)\right) \\ &= \Theta(n^{\log_b a}) + \Theta\left(n^{\log_b a} \sum_{j=0}^{\log_b n - 1} \lg^k n\right) \\ &= \Theta(n^{\log_b a}) + \Theta(n^{\log_b a} \log_b n \lg^k n) \\ &= \Theta(n^{\log_b a}) + \Theta(n^{\log_b a} \lg^{k+1} n) \\ &= \Theta(n^{\log_b a} \lg^{k+1} n) \end{aligned}$$



# نکته‌ای ظریف اما مهم در ساده‌سازی اثبات‌ها

Nevertheless, we must always be on guard when we use asymptotic notation over a limited domain lest we draw improper conclusions. For example, proving that  $T(n) = O(n)$  when  $n$  is an exact power of 2 does not guarantee that  $T(n) = O(n)$ . The function  $T(n)$  could be defined as

$$T(n) = \begin{cases} n & \text{if } n = 1, 2, 4, 8, \dots, \\ n^2 & \text{otherwise,} \end{cases}$$



# روش‌های حل رابطه بازگشتی: درخت بازگشت

فصل ۴.۴ کتاب

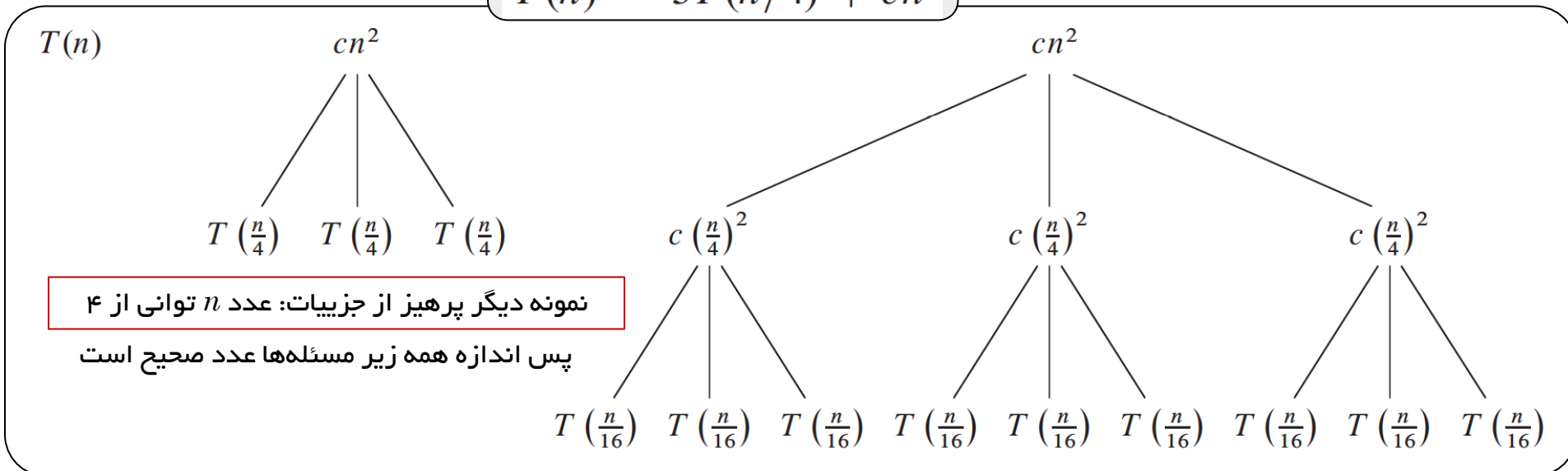
# روش درخت بازگشتی

- روش جایگذاری و استقراء مناسب برای حل روابط بازگشتی
- اما پیدا کردن حدس مناسب همیشه راحت نیست! ← درخت بازگشتی
- در محاسبات درخت بازگشت می‌توان از برخی جزییات پرهیز کرد
- در آن صورت اثبات حدس به دست آمده از طریق استقراء ضرورت خواهد داشت

$$T(n) = 3T(\lfloor n/4 \rfloor) + \Theta(n^2)$$

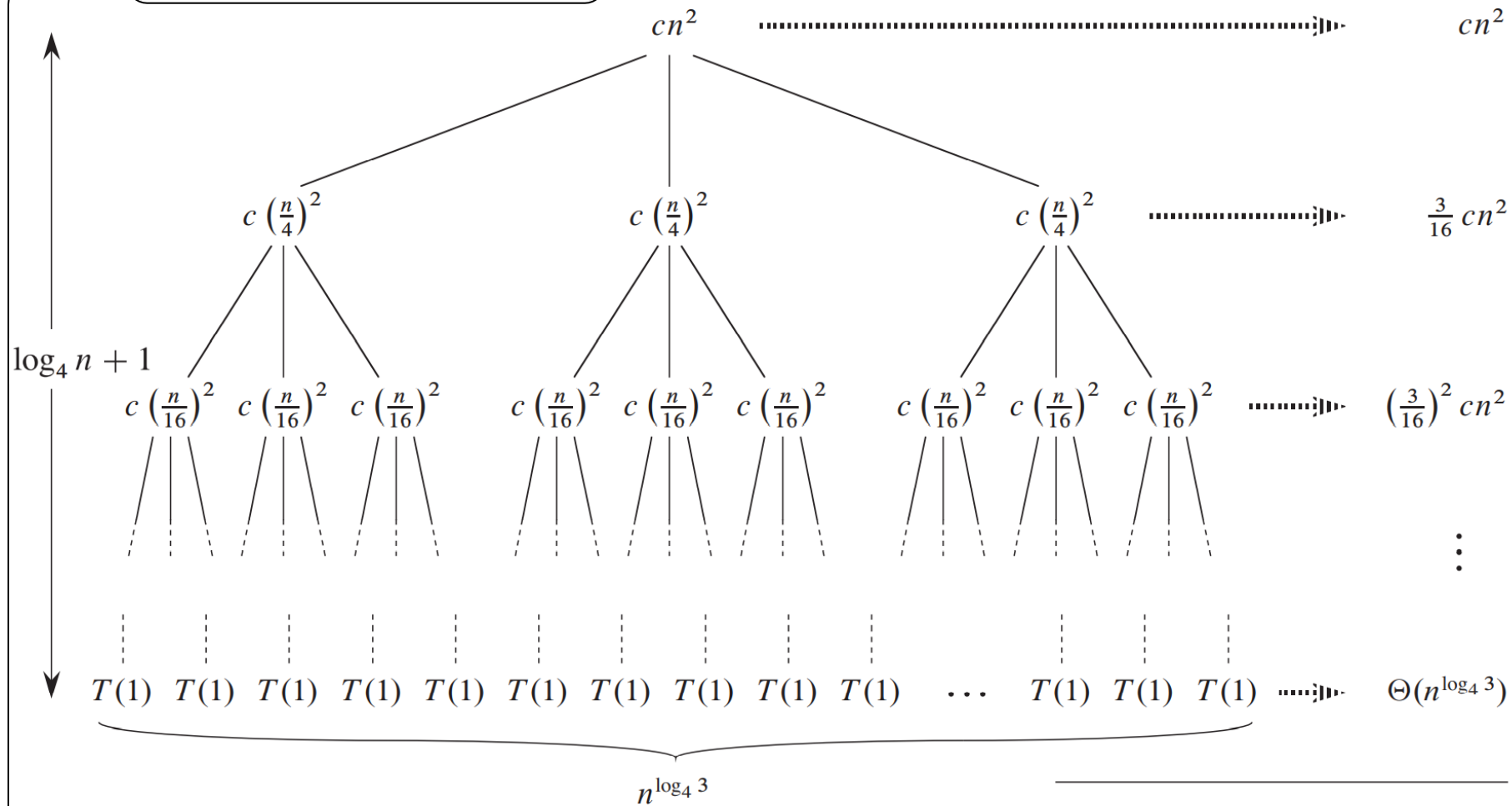
نمونه پرهیز از جزییات: محاسبه حد بالا با حذف کف

$$T(n) = 3T(n/4) + cn^2$$



# روش درخت بازگشتی

$$T(n) = 3T(n/4) + cn^2$$



(d)

Total:  $O(n^2)$

محاسبه مجموع هم سطح‌ها

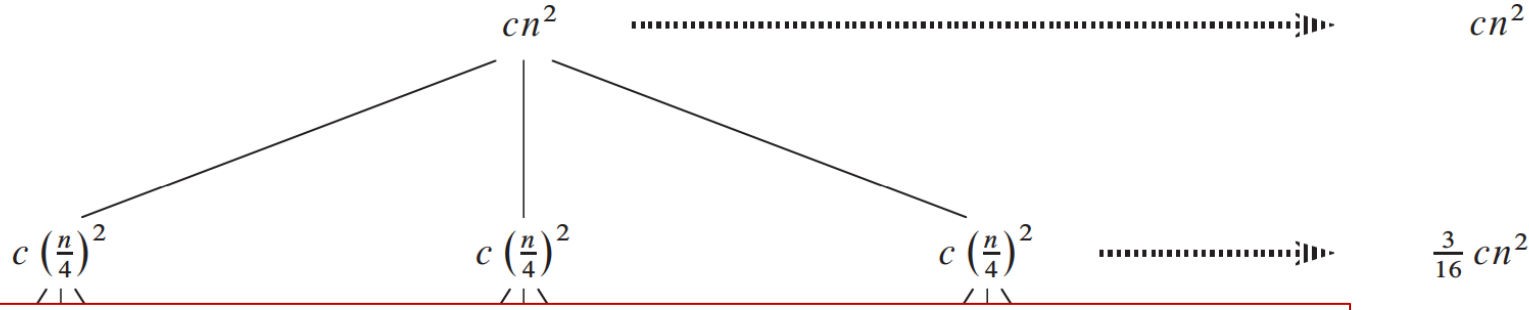
محاسبه تعداد سطوح

$$n/4^i = 1$$

محاسبه مجموع سطوح

# روش درخت بازگشتی

$$T(n) = 3T(n/4) + cn^2$$



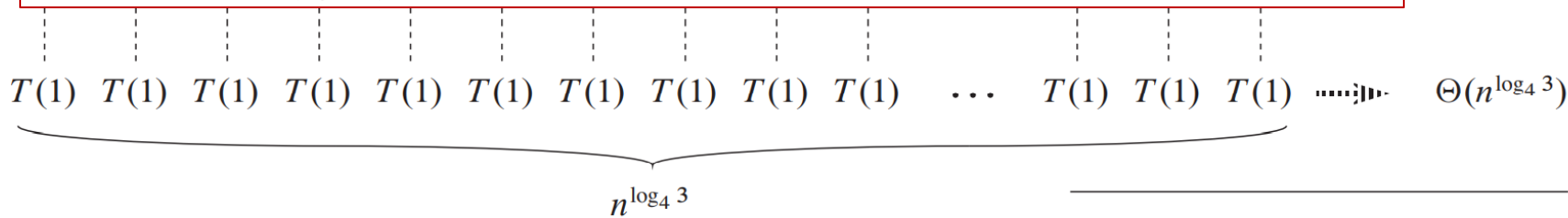
$$T(n) = cn^2 + \frac{3}{16}cn^2 + \left(\frac{3}{16}\right)^2 cn^2 + \dots + \left(\frac{3}{16}\right)^{\log_4 n - 1} cn^2 + \Theta(n^{\log_4 3})$$

$$= \sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3})$$

$$= \frac{(3/16)^{\log_4 n} - 1}{(3/16) - 1} cn^2 + \Theta(n^{\log_4 3})$$

راه ساده تر؟

$$\sum_{k=0}^n x^k = \frac{x^{n+1} - 1}{x - 1}$$



(d)

Total:  $O(n^2)$

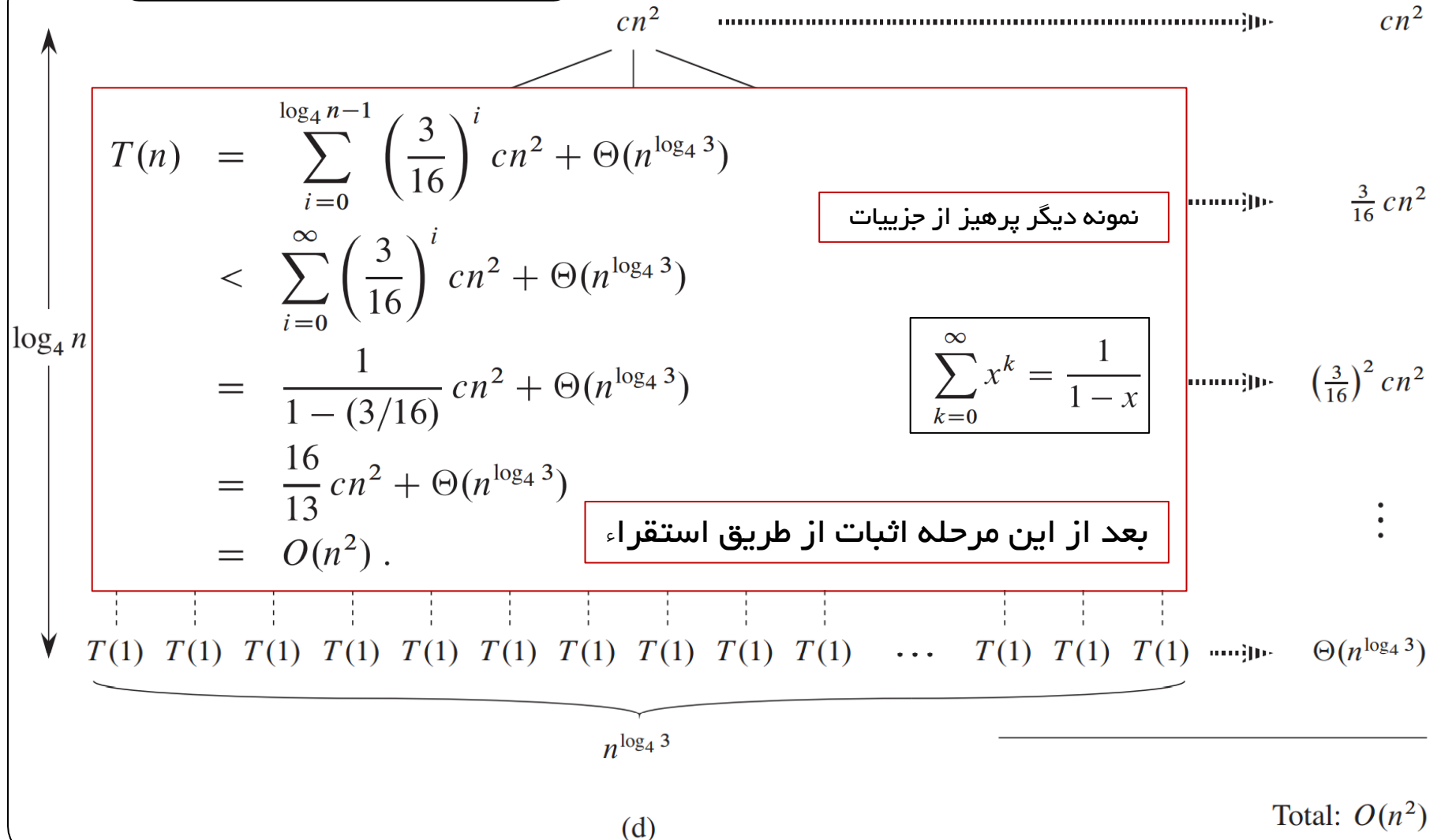
محاسبه مجموع هم سطح‌ها

محاسبه تعداد سطوح

محاسبه مجموع سطوح

# روش درخت بازگشتی

$$T(n) = 3T(n/4) + cn^2$$



نمونه دیگر پرهیز از جزییات

$$\sum_{k=0}^{\infty} x^k = \frac{1}{1-x}$$

بعد از این مرحله اثبات از طریق استقراء

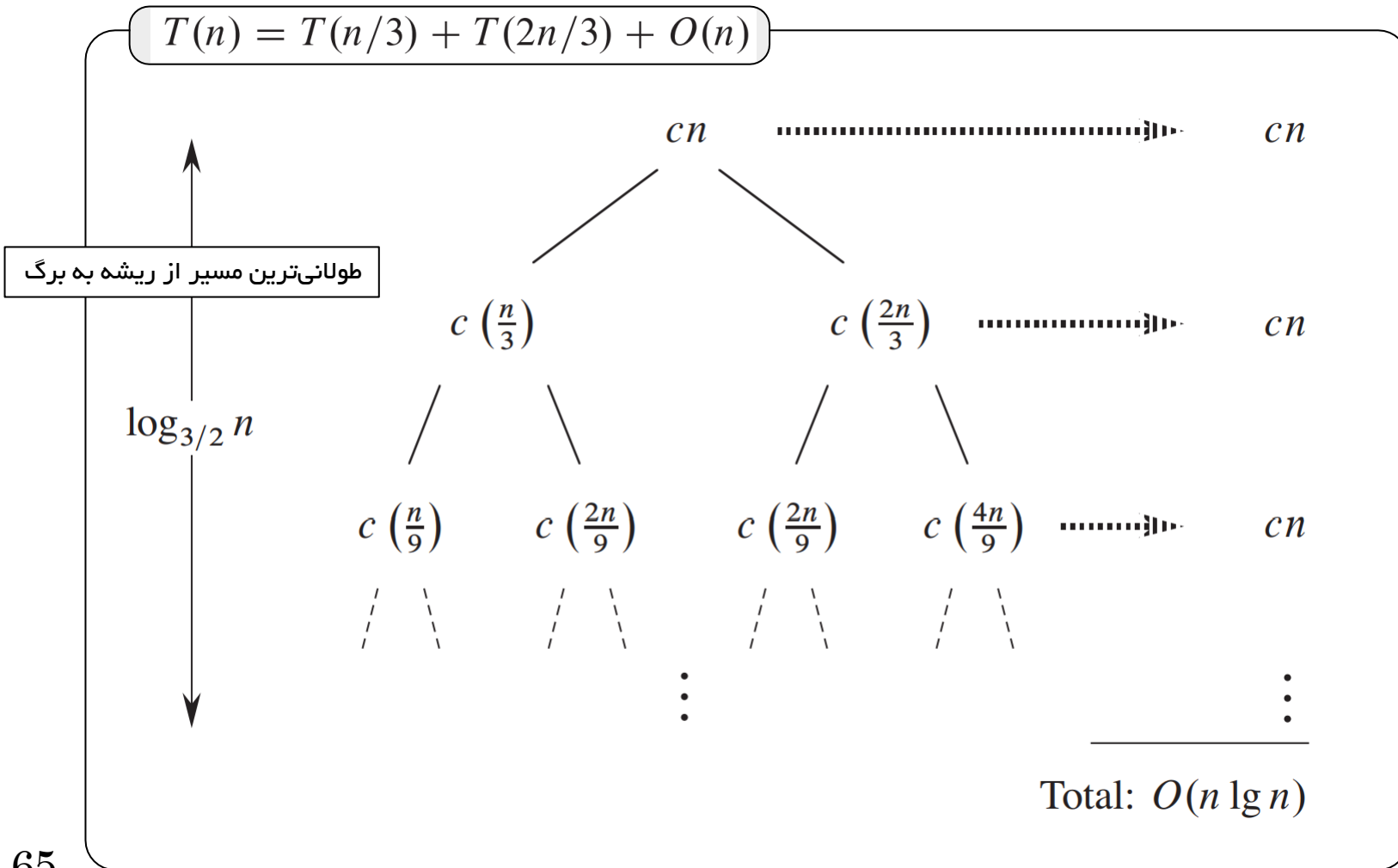
محاسبه مجموع هم سطح ها

محاسبه تعداد سطوح

محاسبه مجموع سطوح

# روش درخت بازگشتی - مثال ۲

$$T(n) = T(n/3) + T(2n/3) + O(n)$$



محاسبه مجموع هم سطوح

محاسبه تعداد سطوح

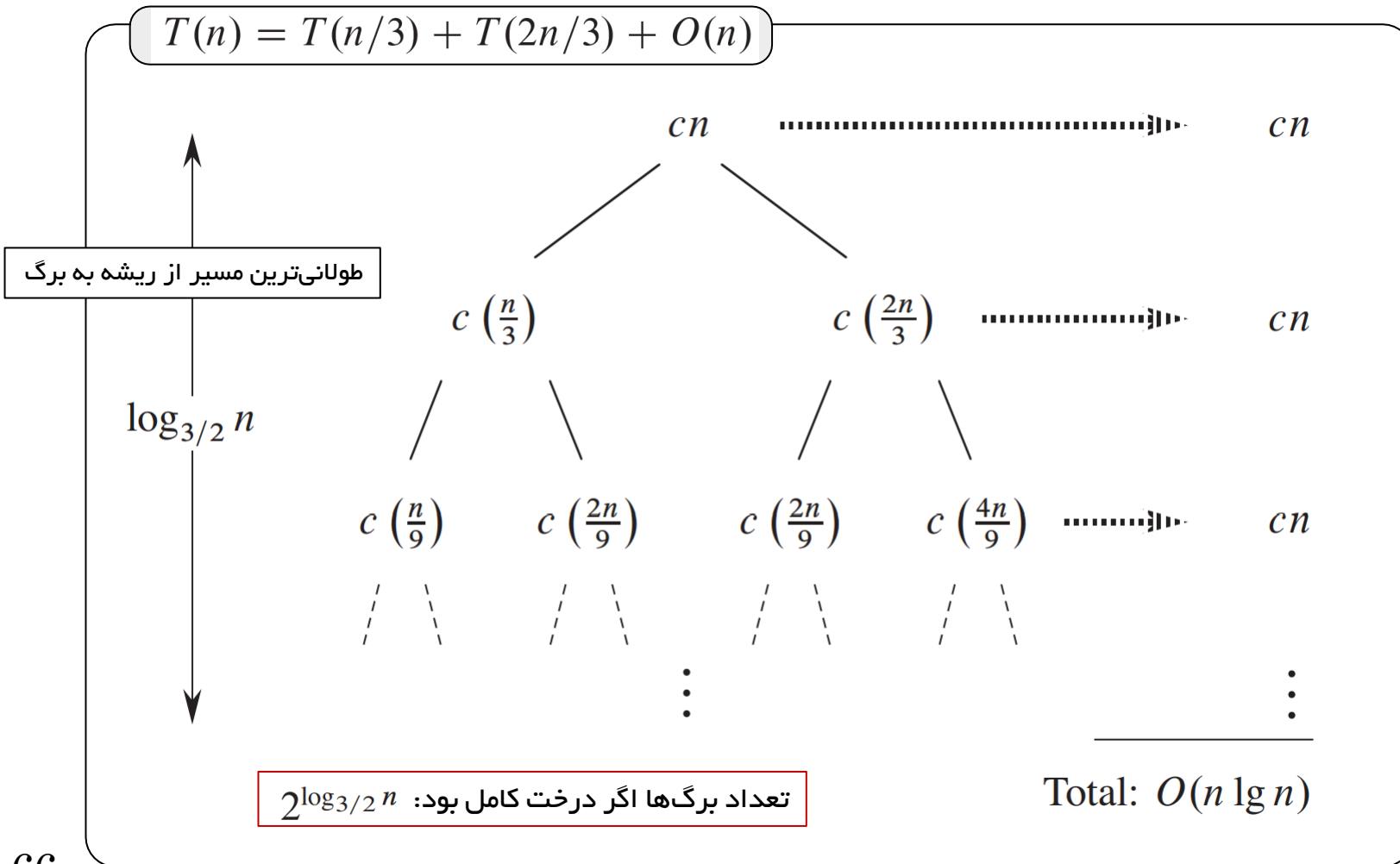
$$(2/3)^k n = 1 \text{ when } k = \log_{3/2} n$$

محاسبه مجموع سطوح



# روش درخت بازگشتی - مثال ۲

$$T(n) = T(n/3) + T(2n/3) + O(n)$$



محاسبه مجموع هم سطح‌ها

آیا درخت کامل است؟

آیا مجموع سطوح همیشه  $cn$  است؟

همه اینها را می‌توان دقیق محاسبه کرد اما...

نمونه‌هایی از پرهیز در محاسبه جزئیات

محاسبه مجموع سطوح

# اثبات حدس روش درخت بازگشتی

$$T(n) = T(n/3) + T(2n/3) + O(n)$$

Total:  $O(n \lg n)$

$$T(n) \leq d n \lg n$$

$$\begin{aligned} T(n) &\leq T(n/3) + T(2n/3) + cn \\ &\leq d(n/3) \lg(n/3) + d(2n/3) \lg(2n/3) + cn \\ &= (d(n/3) \lg n - d(n/3) \lg 3) \\ &\quad + (d(2n/3) \lg n - d(2n/3) \lg(3/2)) + cn \\ &= d n \lg n - d((n/3) \lg 3 + (2n/3) \lg(3/2)) + cn \\ &= d n \lg n - d((n/3) \lg 3 + (2n/3) \lg 3 - (2n/3) \lg 2) + cn \\ &= d n \lg n - d n (\lg 3 - 2/3) + cn \\ &\leq d n \lg n , \end{aligned}$$

$$d \geq c / (\lg 3 - (2/3))$$

اثبات حالت‌های پایه:  
چه بازه ای برای  $d$  در نظر گرفته شود